h_da
HOCHSCHULE DARMSTADT
UNIVERSITY OF APPLIED SCIENCES

Hochschule Darmstadt

– Fachbereich Informatik–

Kohärenzabgleich hierarchischer Zeitreihenprognosen: Evaluation von Network-Coherency-Loss mittels Tourismusdaten und N-BEATS

Abschlussarbeit zur Erlangung des akademischen Grades

Bachelor of Science (B.Sc.)

vorgelegt von

Gibrael Kotwal

Matrikelnummer: 1112193

Referent	:	Prof. Dr. Klaus Kasper
Korreferent	:	Björn Bär

Gibrael Kotwal : *Kohärenzabgleich hierarchischer Zeitreihenprognosen: Evaluation von Network-Coherency-Loss mittels Tourismusdaten und N-BEATS,*
© 02.03.2026

ERKLÄRUNG

Ich versichere hiermit, dass ich die vorliegende Arbeit selbständig verfasst und keine anderen als die im Literaturverzeichnis angegebenen Quellen benutzt habe.

Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder noch nicht veröffentlichten Quellen entnommen sind, sind als solche kenntlich gemacht.

Die Zeichnungen oder Abbildungen in dieser Arbeit sind von mir selbst erstellt worden oder mit einem entsprechenden Quellennachweis versehen.

Diese Arbeit ist in gleicher oder ähnlicher Form noch bei keiner anderen Prüfungsbehörde eingereicht worden.

Darmstadt, 02.03.2026

Gibrael Kotwal

ABSTRACT

Hierarchical time series forecasting requires predictions that are consistent across aggregation levels – a property known as coherence. While classical reconciliation methods enforce coherence as a post-processing step, Hensgen proposed the Network Coherency Loss (NCL), a soft regularization term embedded directly into the loss function of a neural network's final linear layer, enabling coherence to be learned during training.

This work investigates whether NCL is model-agnostic by transferring it to N-BEATS, a purely feed-forward architecture, and evaluating its effect on coherence and forecast accuracy on the Australian Tourism dataset. Experiments across five random seeds show that NCL improves test coherence by 92.4% and test RMSE by 34.0% on average relative to a baseline model trained with MSE loss only. Variance across seeds is also reduced, suggesting increased training stability.

These results support the hypothesis that NCL's effectiveness is not limited to recurrent architectures, provided a linear output layer and a hierarchical summation structure are present. Limitations include the use of a single dataset, a single architecture, and a limited number of seeds, which constrain the generalizability of the findings.

ZUSAMMENFASSUNG

Prognosen für hierarchische Zeitreihen müssen über alle Aggregationsebenen konsistent sein – eine Eigenschaft, die als Kohärenz bezeichnet wird. Klassische Prognoseabgleichmethoden erzwingen Kohärenz als nachgelagerten Schritt. Hensgen schlug demgegenüber die Network Coherency Loss (NCL) vor: einen weichen Regularisierungsterm, der direkt in die Verlustfunktion der letzten linearen Schicht eines neuronalen Netzes eingebettet wird und Kohärenz bereits während des Trainings fördert.

Diese Arbeit untersucht, ob NCL modellagnostisch ist, indem sie NCL auf N-BEATS – eine rein feed-forward-basierte Architektur – überträgt und dessen Wirkung auf Kohärenz und Prognosegüte auf dem Australian-Tourism-Datensatz evaluiert. Experimente über fünf Seeds zeigen, dass NCL die Test-Kohärenz um durchschnittlich 92.4% und den Test-RMSE um 34.0% gegenüber einem Basismodell mit reiner MSE-Verlustfunktion verbessert. Gleichzeitig reduziert sich die Streuung über Seeds, was auf eine erhöhte Trainingsstabilität hindeutet.

Die Ergebnisse stützen die These, dass die Wirksamkeit von NCL nicht auf rekurrente Architekturen beschränkt ist, sofern eine lineare Ausgabeschicht und eine hierarchische Summenstruktur vorliegen. Einschränkend ist anzumerken, dass die Befunde auf einem einzelnen Datensatz, einer einzelnen Architektur und einer begrenzten Anzahl von Seeds beruhen, was die Generalisierbarkeit der Schlussfolgerungen begrenzt.

INHALTSVERZEICHNIS

1 Einleitung 1
1.0.1 Problemstellung 2
1.0.2 Lösungsansätze 2
1.0.3 Einordnung 3
1.0.4 Ziele und Forschungsfragen 4
1.1 Gliederung 4
2 Grundlagen und Theorie 6
2.1 Prognose 6
2.1.1 Prognosearten 6
2.2 Zeitreihen 8
2.2.1 Begriffsintuition und formale Definition 8
2.2.2 Eigenschaften zur Einordnung von Prognoseproblemen 8
2.2.3 Triviale und nicht-triviale Prognoseprobleme 9
2.3 Allgemeine Methoden der Zeitreihenprognose 10
2.3.1 Annahmenbasierte (klassische) Approximatoren 10
2.3.2 Datengetriebene Approximatoren: Künstliche Neuronale Netze 10
2.4 Künstliche Neuronale Netze als Prognosemittel 11
2.4.1 Allgemeine Funktionserklärung 11
2.4.2 Metriken und Verlustfunktion 12
2.5 Kohärenz 16
2.5.1 Grundgedanke der Kohärenzbildung 16
2.5.2 Notation und Summenmatrix-Entwicklung 17
2.5.3 Prognoseabgleichsmethoden 18
2.5.4 Kohärenz durch kleinste Quadrate (OLS-Reconciliation) 19
2.5.5 Bezug zur verwendeten Kohärenzmetrik 20
2.5.6 ANN-basierte Ansätze 20
2.6 Network Coherency Loss 22
2.6.1 Grundgedanke 22
2.6.2 Experimentelle Methodik 24
2.6.3 Kritische Einordnung 25
2.6.4 Implikationen für diese Arbeit 25
3 Methodik 26
3.1 Erwartungen und Eingrenzung 26
3.2 Testumgebung und verwendete Software 27
3.3 Datensatz 27
3.3.1 Datenerläuterung 27
3.3.2 Hierarchiekonstruktion (State, Zone, Region, Purpose) . 28
3.3.3 Datenvorverarbeitung 28
3.4 Trainingsmodell (N-BEATS) 31
3.4.1 Architekturbeschreibung 31
3.4.2 Notation 31

3.4.3 Modellumsetzung und Pipeline 32
3.4.4 Trainingsparameter Übersicht 33
3.4.5 Ausgabelogik 34
4 Ergebnisse 35
4.0.1 Hauptresultate 35
4.0.2 RMSE-Standardabweichung über Seeds 36
4.0.3 Trainingsdynamik 37
4.1 Diskussion 40
4.1.1 Zusammenfassung der Ergebnisse 40
4.1.2 Interner Datenvergleich 41
4.1.3 Externer Vergleich mit Hensgen 41
4.1.4 Bezug auf die Ziele, Fragestellungen und Hypothesen . 42
4.2 Limitierungen 43
5 Fazit 44
5.0.1 Limitierungen 44
6 Weitere Forschung 46

I Appendix
A Anhang 49
A.1 Gitrepository 49
A.2 Rohdaten und weitere Plots 49
A.3 Beispielrechnung der Auswirkung von Normalisierung 50
A.4 Beispielrechnung von NCL 51
A.5 NCL ohne Normalisierung 53

Literatur 54

ABBILDUNGSVERZEICHNIS

Abbildung 2.1 Eine generische Hierarchie, gebildet aus den disaggregierten Knoten AA, AB, AC, BA und BB. Aus [16, Kap. 11.1]. 16
Abbildung 3.1 N-BEATS Blöcke, Blockinhalt und Aufteilung aus [27, S.3] 31
Abbildung 4.1 Exemplarischer Kohärenzverlauf des Basismodells . . . 37
Abbildung 4.2 Exemplarischer Kohärenzverlauf des NCL-Modells . . . 37
Abbildung 4.3 Basis und NCL Validierungskohärenz logarithmisch dargestellt 38
Abbildung 4.4 Validierungs-RMSE des Basismodells über Epochen . . 38
Abbildung 4.5 Validierungs-RMSE des NCL-Modells über Epochen . . 39
Abbildung 4.6 Logarithmisch skalierte Median-Validierungs-RMSE von Basis- und NCL-Modell über Epochen und Läufe 39
Abbildung 4.7 Logarithmische Validierungs-Kohärenz gegen Validierungs-RMSE in Originalwerten 40
Abbildung A.1 Zusammenfassung der Ergebnisse als Mittelwert und Standardabweichung 49
Abbildung A.2 Ergebnisse des Basismodells und des NCL-Modells pro Lauf 49
Abbildung A.3 α pro Epoche 50
Abbildung A.4 Alphaskalierung über alle Seeds 53
Abbildung A.5 Alphaskalierung über Seed 1337 53

TABELLENVERZEICHNIS

Tabelle 2.1 Konzeptioneller Vergleich weicher Kohärenzansätze in neuronalen Modellen. 22

Tabelle 2.2 Relative, prozentuale Verbesserung der gemittelten Daten 25

Tabelle 3.1 Hyperparameter und Protokollparameter der Experimente. 34

Tabelle 4.1 Ergebnis-Zusammenfassung (Mittelwert $\pm$ Standardabweichung über Seeds, Originaleinheiten). $\Delta\%$ ist relative Verbesserung von NCL gegenüber Basismodell. . 35

Tabelle 4.2 Einzelruns pro Seed (Originaleinheiten). Δ in % relativ zum Basismodell: $\Delta[\%] = \frac{B-NCL}{B} \cdot 100$ (positiv = NCL besser). 36

ABKÜRZUNGSVERZEICHNIS

NCL Network Coherency Loss

ANN Artificial Neural Network

RNN Recurrent Neural Network

MLP Multilayer Perceptron

LSTM Long Short-Term Memory

N-BEATS Neural Basis Expansion Analysis for Time Series

MAE Mean Absolute Error

RMSE Root Mean Squared Error

MSE Mean Squared Error

OLS Ordinary Least Squares

MAPE Mean-Absolute-Percentage-Error

sMAPE Symmetric-Mean-Absolute-Percentage-Error

MinT Minimum Trace

1 EINLEITUNG

Im antiken Griechenland wurde das Orakel von Delphi nach einer prognōsis, dem Vorwissen, einer Schlacht oder der Tragweite einer politischen Entscheidung befragt.[5, S.67] Prognosen dienten primär in einer entscheidungsunterstützenden Rolle. Heute (2025) verschiebt sich diese Rolle: Prognosen dienen nicht mehr nur als Orientierung, sondern steuern in vielen Anwendungsfeldern Prozesse direkt.[5, S.6, S.69] Maschinelle Analysen und darauf basierende Prognosen werden zunehmend zu handlungsleitenden Signalen in (teil-)automatisierten Systemen, etwa in der Absatzplanung oder in der Lagerdisposition [5, S.71ff] Mit der wachsenden operativen Bedeutung von Prognosesystemen steigt zugleich der ökonomische Wert ihrer Prognosegüte. Wer Marktbewegungen, Nachfrageschwankungen oder Risiken zuverlässig und präzise vorhersagen kann, gewinnt Handlungsspielraum[23, S.1]: Staaten können Planungsunsicherheiten für den Staatsetat gewinnen[3, S.430ff]; Unternehmen reduzieren Fehler oder verbessern ihren Service.

Ein zentrales Merkmal der genannten Organisationen ist ihre hierarchische Struktur. Dies spiegelt sich auch in ihren internen Daten wider[3, S.430]. Die Daten sind in der Regel Mischungen aus räumlichen Strukturen[8] und zeitlichen Strukturen [3, S.440ff], welche zu Aggregaten zusammengefasst werden können.[1] Für diese Arbeit liegt der Schwerpunkt auf räumlichen Strukturen und deren Aggregaten.

Da in der Praxis Prognosen normalerweise nur auf einer einzelnen Ebene der Hierarchie erstellt werden [8, S.246], [16], stellt sich die nächstliegende Frage, welche dieser Ebenen verwendet werden soll.

Disaggregierte Daten sind Daten, welche aus der untersten Ebene der Hierarchie stammen, z. B. Warenumsätze in einem Zeitraum. Diese enthalten spezifische Informationen, verzerrende Informationen wie Fehlwerte oder Buchungseffekte. Diese Fehler können die Prognosequalität reduzieren, sofern Rohdaten verwendet werden[3, S.433ff]. Aggregationen sind Zusammenfassungen aus disaggregierten Daten, welche über ihnen in der Hierarchie stehen, wie z. B. die Zusammenfassung von Filial- zu Regionalumsätzen. Diese reduzieren den Detailgrad, wirken aber glättend, da sich unabhängige oder gegenläufige Schwankungen teilweise ausmitteln. Dadurch treten längerfristige Trends und saisonale Muster klarer hervor, allerdings auf Kosten der Spezifität [3, S.434], [8, S.241].

Disaggregierte Daten liefern zwar den höchsten Detailgrad, dafür aber auch mehr Fehler. Aggregierte Daten enthalten geglättete langfristige Muster, aber auf Kosten des Detailgrads. So entsteht eine Abwägung zwischen Detailgrad und Zuverlässigkeit der Daten.

1 Anstatt nach Raum und Zeit kann nach Produkt und Zeit gruppiert werden.

1.0.1 *Problemstellung*

Durch die genannten Informationsunterschiede ergibt sich die Motivation, disaggregierte und aggregierte Daten in ihrer Gesamtstruktur zu betrachten, um sowohl kurzfristige, spezifische Signale als auch geglättete, robustere Muster zur Prognosebildung zu verwenden. Dies ist möglich, weil zwischen den Datensätzen eindeutige Aggregationsbeziehungen bestehen. Aggregierte Daten setzen sich aus disaggregierten Daten zusammen, wobei diese Zusammensetzung als lineare Beziehung modelliert werden kann, siehe 2.5.1. Isolierte Einzelprognosen beachten keine hierarchischen Strukturen und damit auch keine Aggregationsbeziehungen. Werden mehrere dieser Einzelprognosen getroffen, können unterschiedliche Prognoseaussagen entstehen, welche zu Konflikten führen. Beispielsweise könnten Beschaffungs- und Lagerentscheidungen auf Filialebene eine andere Gesamtmenge implizieren als Budget- oder Kapazitätsentscheidungen auf Regionsebene. Dadurch entstehen Doppelzählungen, Fehlallokationen oder widersprüchliche Steuerungsgrößen, insbesondere wenn Prognosen direkt in Entscheidungsprozesse einfließen. Dies verringert die Systemeffektivität und untergräbt das Systemvertrauen.

1.0.2 *Lösungsansätze*

Ein naheliegender Lösungsansatz ist der Prognoseabgleich (engl. Forecast Reconciliation[3]): Der Prognoseabgleich hat das Ziel, Prognosen für alle Ebenen einer Hierarchie konsistent zu erstellen. Dies erfolgt durch zusätzliche Informationseinspeisungen aus der hierarchischen Struktur und wird als Kohärenz bemessen. Eine Prognose ist in diesem Sinne kohärent, wenn die Prognosewerte der summierten disaggregierten Daten dem Wert der Prognose des Aggregats entsprechen (vgl.[3], [16], [15],[10], [2]).[2] Traditionelle Prognoseabgleichsmethoden erzwingen Kohärenz als nachgelagerten Abgleich – entweder als Deaggregation der Basisprognose der obersten Ebene oder als Aggregation aus Basisprognosen der disaggregierten Ebenen, wobei die jeweils restlichen prognostizierten Werte missachtet werden[15]. Diese verbessern durch einen nachgelagerten, rigiden Prognoseabgleich die Kohärenz, können aber die Prognosegüte der Basisprognosen zugunsten einer höheren Kohärenz beeinträchtigt [16], [10]. Daraus ergibt sich das Forschungsinteresse: Kohärenz nicht nachträglich zu erzwingen, sondern während der Prognoseerstellung als weiteres Optimierungsziel zu berücksichtigen. Dies geschieht etwa durch das Einbetten von Aggregationsbeziehungen in die Verlustfunktion eines künstlichen neuronalen Netzes (engl. artificial neural network, ANN). Diese werden in Form eines Kohärenzterms neben der Prognosegüte eingefügt. In der Literatur werden solche Prognosemethoden weiche Prognoseabgleichmethoden genannt, vgl. [3, 9, 11].

2 Dies gilt auch für die Summe von Aggregaten, welche zu einem Aggregat zusammengefasst werden.

1.0.3 *Einordnung*

Der Untersuchungsgegenstand dieser Arbeit ist die weiche Prognoseabgleichmethode von Michael Lowell Hensgen für künstliche neuronale Netze. Dieser entwickelte die Netzwerk-Kohärenz-Verlustfunktion (engl. Network Coherency Loss, NCL).

Die von Hensgen entwickelte Verlustfunktion zeigt Verbesserungen sowohl in der Kohärenz als auch in der Prognosegüte gegenüber einem Basismodell ohne NCL und einem Basismodell mit einer nachgelagerten Prognoseabgleichmethode.

So wird eine Kohärenzverbesserung gegenüber einem Basismodell ohne NCL von bis zu 94.93 % und eine Prognoseverbesserung von bis zu 43.23 % festgestellt.

Für seine Ergebnisse verwendet Hensgen ein rekurrentes neuronales Netz in Form eines Long Short-Term Memory [13] Modells, wobei er selbst hinterfragt, ob NCL für jedes andere ANN mit einer letzten linearen Ausgabeschicht verwendet werden kann. Diese Arbeit knüpft an dieser Frage an und testet, ob Kohärenz und Prognosegüte auch mit einem Feed-Forward-Modell verbessert werden können. Hierfür wird Hengsens Versuchsaufbau nachgestellt.

Dazu wird ein neuronales Basisexpansionsanalyse-Modell für Zeitreihen (engl. Neural Basis Expansion Analysis for Time Series, N-BEATS) als alternative Architektur gewählt [27]. N-BEATS ist ein rein feed-forward organisiertes ANN mit einer getrennten letzten linearen Ausgabeschicht, die der von Hensgen formulierten NCL-Definition entspricht. Zudem ist N-BEATS in bestehenden Bibliotheken verfügbar [27] und in der Fachliteratur behandelt [26, 27]. Des Weiteren fand es Verwendung in mehreren Prognosewettbewerben [8, 26, S. 195]. Dies ermöglicht eine transparente und reproduzierbare Implementierung von Modell und NCL auf einer in der Praxis bewährten Architektur.

Als Datengrundlage dient der öffentlich verfügbare und in der Forschung zu hierarchischen Zeitreihen etablierte Australian-Tourism-Datensatz. Er umfasst monatliche Besucherübernachtungen des australischen Inlandstourismus und ist nach geografischen Regionen sowie Reisezwecken hierarchisch strukturiert [2, 38]. Etabliert wurde der Datensatz durch seine wiederholte Nutzung als empirischer Benchmark in Arbeiten zu klassischen Prognoseabgleichmethoden [2, 38] aufgrund seines Realweltbezugs sowie der zugleich handhabbaren Größe und Komplexität [2, 38]. Da der Datensatz auch bei Hensgen Verwendung findet [11], ermöglicht dies einen direkten Vergleich der Wirkung von NCL mit den dort berichteten Ergebnissen. Die Daten sind abrufbar unter [32].

1.0.4 *Ziele und Forschungsfragen*

Die Arbeitsziele sind wie folgt:

1. Übertragung von NCL auf ein Feed-Forward-basiertes Modell mit den von Hensgen verwendeten Tourismusdaten, um eine mögliche Modellagnostik der NCL-Wirkung zu stützen oder kritisch einzuordnen.
2. Analyse der Wirkung von NCL auf die Kohärenz unter Betrachtung der Prognosegüte.

Aus diesen Zielen werden die folgenden Forschungsfragen abgeleitet:

F1: Verbessert NCL die hierarchische Kohärenz gegenüber einem Kontrollmodell ohne NCL, und wenn ja, in welchem Ausmaß?

F2: Wie beeinflusst NCL die Prognosegüte hinsichtlich der bei Hensgen verwendeten Metriken gegenüber einem Kontrollmodell ohne NCL?

F3: Wie unterscheiden sich die Ergebnisse eines Feed-Forward-basierten Modells von denen des bei Hensgen verwendeten rekurrenten neuronalen Modells?

MEHRWERT DER ARBEIT Diese Arbeit soll eine reproduzierbare, klar dokumentierte Testbank für Kohärenzregularisierungsfunktionen wie NCL bilden, um auf dieser Grundlage weitere Forschungen betreiben zu können. Des Weiteren soll diese Arbeit die Übertragbarkeit von NCL auf andere ANN-Architekturen untersuchen.

1.1 GLIEDERUNG

Die Arbeit gliedert sich wie folgt: Kapitel 2 legt die begrifflichen und methodischen Grundlagen für Prognosen und Zeitreihen sowie für hierarchische Zeitreihensysteme und Kohärenz. Darauf aufbauend werden klassische Prognoseabgleichmethoden (insb. OLS / MinT als Projektion in den kohärenten Raum) sowie neuronale Netzwerkansätze für Prognosen eingeordnet. Den Abschluss des Kapitels bildet die Herleitung und Einordnung der *Network Coherency Loss* nach Hensgen einschließlich der zentralen Annahmen, der experimentellen Referenzmethodik und der Implikationen für diese Arbeit.

Kapitel 3 beschreibt die Methodik der empirischen Untersuchung. Hierzu gehören die Testumgebung und Softwarebasis, der verwendete Datensatz sowie die Konstruktion der Hierarchie (State, Zone, Region, Purpose) und die Datenvorverarbeitung. Anschließend wird das Trainingsmodell (N-BEATS) inklusive Notation, Pipeline und Ausgabelogik spezifiziert. Abschließend werden die Trainingsparameter, die verwendeten Evaluationsmetriken und insbesondere die Kohärenzmetrik als Abweichungsmaß formal definiert, sodass die Experimente reproduzierbar sind.

Kapitel 4 präsentiert die Ergebnisse der Experimente und bereitet sie strukturiert auf. Neben den Hauptresultaten werden die Robustheit der Prognosegüte über Seeds, die Trainingsdynamik, sowie der Verlauf der Regularisierung (Alpha-Verlauf) analysiert und durch Zwischenfazits zusammengeführt. Anschließend werden die Resultate diskutiert im Hinblick auf die Forschungsfragen. Dazu werden Basismodell und NCL gegenübergestellt und die Befunde mit Hengsens Referenzergebnissen verglichen. Abschließend werden Limitationen der Studie herausgearbeitet.

Kapitel 5 fasst die Arbeit im Fazit zusammen und ordnet die Ergebnisse hinsichtlich Prognosegüte und Kohärenz ein.

Kapitel 6 fast weitere Forschungsideen auf.

Appendix/Anhang dokumentiert die konkret verwendeten Implementierungsdetails, Beispiele, Graphen die nicht in die Hauptuntersuchungen gekommen sind, Rohdaten und Link zum Github-Repository mit der Implementierung und Containerisierung zur Nachvollziehbarkeit und Transparenz.

2 GRUNDLAGEN UND THEORIE

Das Grundlagenkapitel ist wie folgt gegliedert: Zunächst werden zentrale Begriffe dieser Arbeit - insbesondere Prognosen, Zeitreihen und neuronale Netze - definiert und voneinander abgegrenzt. Anschließend werden die verwendeten Bewertungsmetriken eingeführt. Darauf aufbauend wird das Kohärenzproblem in hierarchischen Zeitreihen formalisiert. Im weiteren Verlauf wird der Forschungsstand dargestellt, beginnend bei klassischen Verfahren des Prognoseabgleichs und übergehend zu ANN-basierten Ansätzen. Abschließend wird Hengsens Arbeit vorgestellt, einschließlich der Netzwerk-Kohärenz-Verlustfunktion, relevanter Begriffe und der berichteten experimentellen Ergebnisse.

2.1 PROGNOSE

Dieser Abschnitt führt den Begriff der Prognose ein und grenzt ihn im Kontext der Statistik und Zeitreihenanalyse auf die in dieser Arbeit verwendete Definition einer zeitreihenbasierten Punktprognose ein.

2.1.1 *Prognosearten*

Allgemein wird die Prognose wie folgt definiert: Laut Duden [6] ist eine Prognose eine „wissenschaftlich begründete Voraussage einer künftigen Entwicklung, künftiger Zustände, des voraussichtlichen Verlaufs". Das Gabler Wirtschaftslexikon definiert Prognose als „Aussage über zukünftige Ereignisse, bezüglich zukünftige Werte [...], beruhend auf Beobachtungen aus der Vergangenheit und auf theoretisch fundierten objektiven Verfahren"[39]. Prognosen können nach ihrer Analysemethode taxonomisch unterschieden werden. Dies dient dazu, im folgenden Abschnitt den Untersuchungsumfang der Arbeit einzugrenzen.

Laut Vogel [36, S. 11ff.] werden Prognosen nach ihrem Prognoseansatz in qualitative (auch subjektive [5, S. 90ff.]) und quantitative (auch objektive [5, S. 90ff.]) Prognosen unterschieden. Qualitative Prognosen basieren auf subjektiven Erwartungen und Einschätzungen [16, Kap. 1.4]. Demgegenüber steht die quantitative Prognose: Sofern numerische Werte verfügbar sind und angenommen werden kann, dass die darin enthaltenen Muster auch zukünftig fortbestehen, können rechenbasierte Methoden angewandt werden [16, Kap. 1.4].

Die quantitative Prognosebildung wird in zwei Klassen eingeteilt.

Die Wirkungsprognose bildet die Analyse von Maßnahmen und Entscheidungsprozessen ab, bei denen ein Eingriff in eine bedingte Zukunft möglich ist [5, 36, S. 13]. Diese Prognoseart setzt voraus, dass Effekte und deren

Wirkungen bekannt sind. Gegen die Verwendung der Wirkungsprognose spricht die Schwierigkeit der Realitätsmodellierung und der darin enthaltenen Interdependenzen, welche zur Vereinfachung als lineare Beziehungen angenommen werden 2.5.1.

Die Entwicklungsprognose hingegen behandelt Effekte und Wirkungen als unbekannte Variablen. Die vorherzusagende Zielgröße wird als nicht direkt beeinflussbar angenommen [5, S. 80]. Wird der Interessengegenstand explizit in Abhängigkeit von der Zeit betrachtet, spricht man von einer *Zeitreihenanalyse* [36, Abb. 2.2][5, S. 80ff.]. Diese Arbeit beschränkt sich auf die Zeitreihenprognose, da numerische Verfahren auf Basis von Zeitreihendaten untersucht werden.

PROGNOSEZIELE werden für diese Arbeit klassifiziert nach Prognosehorizont und Prognosegranularität.

Der Prognosehorizont beschreibt, wie weit im Voraus prognostiziert wird. Technisch wird er mit h abgekürzt und entweder iterativ oder direkt berechnet. Für diese Arbeit wird eine *1-Step-Ahead*-Direktprognose gewählt, da diese in der Literatur üblicherweise zu Vergleichszwecken verwendet wird [5, 8, S. 58]. Die 1-Step-Ahead-Direktprognose prognostiziert auf Basis historischer Werte von einem Zeitpunkt t auf einen Zeitpunkt $t + 1$. Weiterschreibende Prognosen existieren ebenfalls (vgl. [4, 5][25, S. 68ff.]), fallen jedoch aus dem Betrachtungsumfang dieser Arbeit.

Die Prognosegranularität wird für diese Arbeit als Unterscheidung zwischen Intervall- bzw. probabilistischen Prognosen und Punktprognosen verstanden [3, 5][8, Kap. 3.2]. Während eine Punktprognose pro Prognosezeitpunkt einen einzelnen numerischen Schätzwert liefert, drücken Intervallprognosen die Unsicherheit explizit als Vorhersageintervall aus. Probabilistische Prognosen verallgemeinern dies zu einer vollständigen Wahrscheinlichkeitsverteilung [16]. Vorhersageintervalle und Verteilungsprognosen erfordern zusätzliche Modellannahmen sowie Verfahren, die Unsicherheit explizit modellieren [16]. Da eine Punktprognose nur einen einzelnen Schätzwert liefert, enthält sie keine explizite Unsicherheitsinformation und ist einfacher zu modellieren, zu interpretieren und erfordert weniger Daten [8, S. 64].

Im Rahmen dieser Arbeit wird das Prognoseziel deshalb bewusst als *zeitreihenbasierte 1-Step-Ahead-Direktprognose* festgelegt und mit Punktfehlmaßen bewertet, um den Einfluss der Kohärenzregularisierung auf die Prognosegenauigkeit und Kohärenz möglichst isoliert untersuchen zu können [16].

2.2 ZEITREIHEN

Basierend auf Vogel [36, S. 19ff.] und Crone [5, S. 104] wird der Begriff der Zeitreihe präzisiert, um Prognoseprobleme anhand von Zeitreiheneigenschaften einzuordnen und geeignete Prognoseverfahren abzuleiten.

2.2.1 *Begriffsintuition und formale Definition*

Intuitiv ist eine Zeitreihe eine Folge von Beobachtungen einer Interessengröße, die über aufeinanderfolgende Zeitpunkten erhoben wird. Formal wird dabei zwischen dem *stochastischen Prozess* und der *beobachteten Realisierung* unterschieden [5, 36, Kap. 3ff.][23, S. 3].

STOCHASTISCHER PROZESS Eine Zeitreihe wird als stochastischer Prozess

$$\{Y_t\}_{t\in\mathcal{T}} \tag{2.1}$$

verstanden, wobei jedes Y_t eine Zufallsvariable (oder einen Zufallsvektor) zum Zeitpunkt t bezeichnet. Der Indexraum $\mathcal{T}$ kann kontinuierlich ($\mathcal{T} \subseteq \mathbb{R}$) oder diskret ($\mathcal{T} \subseteq \mathbb{Z}$) sein [36, Kap. 3.1]. Da praxisnahe Daten typischerweise diskrete Zeitreihen abbilden, beschränkt sich diese Arbeit auf diskrete Zeitreihen.

WERTEBEREICH / DIMENSION. Zur Abbildung mehrerer Größen wird Y_t als Vektor modelliert:

$$Y_t \in \mathbb{R}^d, \tag{2.2}$$

wobei d die Anzahl der gleichzeitig betrachteten Komponenten beschreibt.

BEOBACHTETE REALISIERUNG. Die tatsächlich vorliegenden Zeitreihen sind eine Realisierung eines stochastischen Prozesses:

$$\{y_t\}_{t\in\mathcal{T}} \quad \text{mit} \quad y_t \in \mathbb{R}^d. \tag{2.3}$$

Für $d = 1$ ergibt sich die univariate (eindimensionale) Zeitreihe $y_t \in \mathbb{R}$ [8, S. 56]; für $d > 1$ die multivariate (mehrdimensionale) Zeitreihe $y_t = (y_{t,1}, \ldots, y_{t,d})^\top$ [8, S. 56] (vgl. [36, S. 19ff.][8, Kap. 5.1ff.]). Für diese Arbeit wird von multivariaten realisierten Zeitreihen ausgegangen, da diese mehr Dimensionen und damit eine größere Bandbreite an Zeitreihenarten abdecken.

2.2.2 *Eigenschaften zur Einordnung von Prognoseproblemen*

Die in 2.2.1 verwendete Zeitreihendefinition umfasst auch deterministische Folgen (z. B. überlagerte Sinuskurven), die bei bekannter Erzeugungsregel prinzipiell exakt fortgeschrieben werden können. Für die Auswahl geeigneter Prognoseverfahren ist daher zu präzisieren, welche Eigenschaften rea-

le Zeitreihen typischerweise *nicht-trivial* machen. In dieser Arbeit werden Zeitreihen entlang der folgenden Merkmale charakterisiert:

- **Stochastizität / Rauschen:** Reale Zeitreihen enthalten zufällige Schwankungen, die nicht vollständig durch eine deterministische Struktur erklärt werden können. Zeitreihen ohne ausnutzbare systematische Struktur werden häufig als (nahezu) weißes Rauschen modelliert [8, Kap. 7.5].
- **Zeitliche Abhängigkeit:** Zeitliche Abhängigkeiten werden über Autokorrelationen zwischen y_t und verzögerten Werten y_{t-k} beschrieben. Sie manifestieren sich empirisch u. a. als Trend- und Saisonkomponenten oder als Persistenz bzw. Trägheit im Niveau [8, S. 56ff.][5, 16, 23].
- **Nichtstationarität:** Änderungen in Erwartungswert, Varianz oder Autokorrelationsstruktur (z. B. Trends, saisonale Muster, Niveauverschiebungen) verletzen Stationaritätsannahmen und motivieren Transformationen (z. B. Differenzenbildung, Varianzstabilisierung) sowie eine angepasste Modellwahl [8, S. 56].
- **Strukturbrüche / Regimewechsel:** Zeitreihen können Bruch- bzw. Wechselpunkte aufweisen, nach denen sich Dynamik, Mittelwert oder Varianz dauerhaft oder temporär ändern. Solche Regimewechsel erschweren die Extrapolation aus der Vergangenheit und beeinflussen Trainings- und Validierungsstrategien [16, 23].
- **Varianzinstabilität und Ausreißer:** Zeitlich variierende Streuung (Heteroskedastizität) kann Schätzung, Skalierung und Verlustfunktionen beeinflussen [23]. Ausreißer treten in Realdaten häufig auf (z. B. Messfehler, Sonderereignisse) und können Prognosen sowie Fehlermaßen stark verzerren [8].
- **Datenqualität:** Fehlende Werte, unregelmäßige Beobachtungen, Kalender- und Buchungseffekte sowie Messfehler sind in praktischen Anwendungen verbreitet; der Umgang damit ist Teil des Prognoseprozesses [8]. Sehr kurze Historien begrenzen zudem die identifizierbare Komplexität und erhöhen das Overfitting-Risiko [16].
- **Hierarchie:** Bei hierarchischen Zeitreihen bestehen Aggregationsbeziehungen zwischen mehreren gekoppelten Reihen. Dies führt zu Kohärenzbedingungen (Aggregate müssen konsistent sein) und motiviert Prognoseabgleich bzw. kohärenzerzwingende Lernziele [16].

2.2.3 *Triviale und nicht-triviale Prognoseprobleme*

Damit einfache, deterministische Folgen von realweltlichen Prognoseaufgaben abgegrenzt werden können, gilt ein Prognoseproblem in dieser Arbeit als *trivial*, wenn die Folge durch eine einfache deterministische Regel mit

konstanten Parametern beschrieben werden kann und keine der oben genannten Komplexitäten, wie Strukturbrüche, zeitvariierende Varianz, Mehrfachsaisonalität oder hierarchische Kopplungen, wesentlich auftritt. Als *nicht-trivial* werden dagegen Zeitreihen verstanden, bei denen stochastische Schwankungen und/oder nichtstationäre bzw. gekoppelte Strukturen die Generalisierung über die Zeit erforderlich machen und die Wahl des Prognoseverfahrens im wesentlichen beeinflussen.

Beispiele für nicht-triviale Zeitreihendatensätze finden sich in der Literatur und aus Prognosewettkämpfen (vgl. [2, 3, 12, 36]). Unter anderem wurden diese auch nach ihrer Realitätstreue diskutiert (vgl. [34]).

2.3 ALLGEMEINE METHODEN DER ZEITREIHENPROGNOSE

Eine 1-Step-Ahead-Direktprognose lässt sich allgemein als Funktionsapproximation formulieren [7, S. 122]: Aus den letzten p Vergangenheitswerten wird ein Modell f gelernt, das den nächsten Wert $\hat{y}_{t+h}$ für den Prognosehorizont $h = 1$ schätzt.

$$\hat{y}_{t+h} = f\left(y_t, y_{t-1}, \ldots, y_{t-p+1}\right)$$

Prognosemethoden unterscheiden sich vor allem darin, *wie stark die Form von f festgelegt ist* [5, S. 81] und *welche Annahmen über die Daten* getroffen werden [8, 16].

2.3.1 *Annahmenbasierte (klassische) Approximatoren*

Klassische statistische Verfahren legen die Modellform weitgehend fest. Dadurch sind sie oft stabil, schnell und gut interpretierbar [16, 23, 36]. Damit diese Modelle einsetzbar sind, werden Strukturannahmen benötigt: etwa dass die Zeitreihe nach Transformationen und Differenzierungen ausreichend stationär ist, Zusammenhänge überwiegend linear bzw. durch wenige Parameter beschreibbar sind [8, S. 137], und Fehlerterme gering sind, d. h. wenig Autokorrelation und näherungsweise konstante Varianz aufweisen [16, 23, 36]. In realen Anwendungen sind diese Voraussetzungen oft nur näherungsweise erfüllt – etwa bei Nichtlinearitäten, Strukturbrüchen oder wechselnder Varianz. Dadurch steigt der Aufwand für manuelle Modellierung (Datenselektion, Transformationen, Feature-Engineering), und die Prognosequalität kann abnehmen [8].

2.3.2 *Datengetriebene Approximatoren: Künstliche Neuronale Netze*

Neuronale Netze verfolgen den gegenteiligen Ansatz. Die Modellform wird nicht von vornherein eingeschränkt, sondern aus Daten gelernt. Theoretisch motiviert wird diese Flexibilität durch das Universal-Approximation-Theorem: hinreichend große Feedforward-Netze können Funktionen beliebig genau approximieren[14, Theorem 2.2], [5, S.207], [7, S.194]. Für Prognoseaufgaben

ist dies relevant, da komplexe und nicht-lineare Muster ohne vollständige manuelle Spezifikation gelernt werden können. [5], [8, S.233ff]. Dies wird auch durch die Literatur bestätigt. ANN-Modelle gelten als wettbewerbsfähige Alternativen zu klassischen Verfahren (vgl.[3],[5],[4],[8],[13]) insbesondere bei komplexen Dateneigenschaften und großen Datensätzen. Kritisch zu betrachten ist, dass diese keine „Selbstläufer" sind. Die Prognosegüte hängt unter anderem von der Datenvorverarbeitung wie Skalierung/Normalisierung ab, sowie der Wahl der Verlustfunktion. [35],([20][1]),[12]

Klassische, annahmenbasierte Modelle sind effizient und interpretierbar, stoßen aber bei nichtlinearen und wechselhaften Zeitreihen an Grenzen. Für die in dieser Arbeit betrachteten Zeitreihen ist ein flexibler, datengetriebener Funktionsapproximator zweckmäßig. Aus diesem Grund werden im Folgenden Artificial Neural Networks (ANNs) als Prognosemodell eingesetzt [5, S. 115, 121][3, 16].

2.4 KÜNSTLICHE NEURONALE NETZE ALS PROGNOSEMITTEL

Künstliche neuronale Netze (engl. *artificial neural networks*, ANN) sind lernfähige Funktionsapproximatoren, die eine parametrisierte Abbildung zwischen Eingaben und Ausgaben modellieren und deren Parameter aus Daten angepasst werden [5, Kap. 4][4, S. 255ff.][16].

2.4.1 *Allgemeine Funktionserklärung*

Basierend auf Crone [5, S. 164ff.], Haben [8, S. 171ff.] und [7] funktioniert ANN folgendermaßen: Ein neuronales Netz beschreibt eine parametrisierte Funktion $f_{\boldsymbol{\theta}}(\cdot)$, die aus Bausteinfunktionen (lineare Transformationen und Aktivierungsfunktionen) zusammengesetzt wird. Ein Beispiel ist das Mehrschichtige Perzeptron (engl. *Multi-Layer Perceptron*, MLP): Eine Eingabe $\boldsymbol{x} \in \mathbb{R}^d$ wird schichtweise transformiert, z. B.

$$\boldsymbol{h}^{(l)} = \sigma\big(\boldsymbol{W}^{(l)}\boldsymbol{h}^{(l-1)} + \boldsymbol{b}^{(l)}\big), \qquad \hat{\boldsymbol{y}} = \boldsymbol{W}^{(L)}\boldsymbol{h}^{(L-1)} + \boldsymbol{b}^{(L)},$$

wobei $\sigma(\cdot)$ eine nichtlineare Aktivierung ist und $\boldsymbol{\theta} = \{\boldsymbol{W}^{(l)}, \boldsymbol{b}^{(l)}\}_l$ die zu lernenden Parameter umfasst [4, S.255ff].

LERNEN ALS OPTIMIERUNGSPROBLEM / GRADIENTENVERFAHREN Das Training eines neuronalen Netzes erfolgt typischerweise als Minimierung eines Fehlers gegenüber den Trainingsbeispielen $(\boldsymbol{x}_i, \boldsymbol{y}_i)$:

$$\min_{\boldsymbol{\theta}} \frac{1}{N} \sum_{i=1}^{N} \ell(\boldsymbol{y}_i, f_{\boldsymbol{\theta}}(\boldsymbol{x}_i)) \,.$$

Wobei $\ell(\cdot,\cdot)$ eine Minimierungs-/Verlustfunktion ist. Die Gradienten von ℓ bezüglich $\boldsymbol{\theta}$ werden mittels Rückwärtspropagierung (eng. Backpropagation)

1 vgl. Kim et al. für den Effekt von Normalisierungstermen auf rekurrente neuronale netzwerke

berechnet, und die Parameter werden iterativ mit (stochastischen) Gradientenverfahren aktualisiert [4]. Damit wird der Zusammenhang zwischen *Trainingsziel* (eng. Loss) und *Prognosequalität* (Evaluationsmetriken) hergestellt. Diese Unterscheidung wird in Abschnitt 2.4.2 präzisiert.

ARCHITEKTURFAMILIEN FÜR PROGNOSEN. Für Zeitreihenprognosen werden unterschiedliche Architekturen eingesetzt. Feed-Forward-basierte Modelle wie das MLP sind konzeptionell einfach, benötigen jedoch häufig ein geeignetes Kontextfenster und eine Merkmalskonstruktion (z. B. Verzögerungen, saisonale Indikatoren), um Zeitabhängigkeiten abzubilden [4]. *Recurrent Neural Networks* (RNN) erweitern dies durch rekursive Zustände und sind darauf ausgelegt, Sequenzen direkt zu verarbeiten. Varianten wie *Long Short-Term Memory* (LSTM) adressieren dabei u. a. Gradientenstabilitätsprobleme bei langen Abhängigkeiten [13]. Daneben werden Faltungsnetze (eng. Convoluted neural networks) und aufmerksamkeitsbasierte Modelle (insb. Transformer-Varianten) genutzt, um längere Kontextfenster und flexible Abhängigkeitsstrukturen effizienter zu modellieren. [10, S.2]

VORTEILE, GRENZEN UND BEZUG ZUR PROBLEMSTELLUNG. Nach Crone [5, S. 211ff.] sind ANN-gestützte Prognosen attraktiv, wenn viele Serien und Merkmale vorliegen, nichtlineare Effekte zu erwarten sind und eine gemeinsame Modellierung (globales Modell über viele Serien) Skaleneffekte erlaubt. Dem stehen u. a. höherer Daten- und Rechenbedarf, Modellkomplexität sowie eingeschränkte Interpretierbarkeit gegenüber. Für hierarchische Zeitreihen ist zudem relevant, dass unabhängig trainierte oder frei prognostizierende Modelle im Allgemeinen *inkohärente* Vorhersagen liefern können (z. B. passt die Summe der Filialprognosen nicht zur Regionsprognose). Diese Beobachtung motiviert die in dieser Arbeit untersuchten Mechanismen zur weichen Kohärenzherstellung direkt im Lernprozess von ANN-Modellen [15].

2.4.2 *Metriken und Verlustfunktion*

Evaluationsmetriken quantifizieren die Abweichung zwischen vorhergesagten Punktwerten $\hat{y}_{t+h}$ und realisierten Beobachtungen y_t. Sie dienen der Modellwahl, dem Hyperparameter-Tuning und der Beurteilung der Generalisierungsfähigkeit und reduzieren damit das Risiko einer Überanpassung an Trainingsdaten [4, S. 255ff.][5, S. 129ff.]. Davon abzugrenzen sind Verlustfunktionen, die im Training minimiert werden, um Modellparameter mittels Gradientenverfahren zu optimieren. In vielen Prognoseaufgaben wird dieselbe Fehlerfunktion sowohl als Trainingsverlust als auch als Berichtmetrik verwendet, wobei die Anforderungen an beide Rollen unterschiedlich sind [35, S. 2].

KLASSIFIKATION GÄNGIGER FEHLERMETRIKEN Terven et al. ordnen Fehlmaße in Familien ein, die sich konzeptionell vor allem durch Skalierung,

Interpretierbarkeit, Robustheit und Vergleichbarkeit unterscheiden.[35][16] Für Zeitreihenprognosen sind insbesondere folgende vier Familien verbreitet:

- **Skalenabhängige Fehlermaße** messen den Fehler auf der Originalskala der Zielgröße. Das erleichtert die fachliche Interpretation der Prognose. Gleichzeitig sind Werte zwischen Zeitreihen mit unterschiedlichen Einheiten oder Größenordnungen nicht direkt vergleichbar. In hierarchischen Daten dominieren zudem große Aggregate häufig die Gesamtbewertung, wenn Fehler ungeeignet aggregiert werden [16]. Typische Vertreter sind der mittlere absolute Fehler (MAE), der mittlere quadrierte Fehler (MSE) und dessen Wurzel (RMSE).

- **Prozentuale und relative Fehlermaße** beziehen den Fehler auf die Größe der Beobachtung und sind damit in vielen Fällen dimensionslos. Das kann Vergleiche über Zeitreihen hinweg vereinfachen. In der Praxis sind sie jedoch problematisch, wenn y_t nahe null liegt, als negativer Wert vorliegt oder Zeitreihen sporadische Beobachtungen aufweisen. Dann entstehen Instabilitäten und Verzerrungen, die die Modellbewertung dominieren können [16]. Typische Vertreter sind der *Mean Absolute Percentage Error* (MAPE) und der *Symmetric Mean Absolute Percentage Error* (sMAPE).

- **Relative Fehlermaße gegenüber Referenzmodellen** setzen den Prognosefehler ins Verhältnis zu einem Referenzmodell, häufig einer klassischen Approximation (vgl. Abschnitt 2.3.1). Dadurch entsteht eine unmittelbare Aussage zur praktischen Nützlichkeit. Der Nachteil liegt in der Abhängigkeit von der Definition des Referenzmodells, die je nach Modell, Aggregationsniveau und Datenqualität variieren kann. Außerdem sind solche Quotienten als Trainingsziel meist ungeeignet, da sie numerisch instabil sein können [16, 35].

- **Skalierte Fehlermaße** normalisieren den Fehler über einen internen Skalierungsfaktor, typischerweise den Fehler einer klassischen Approximation. Damit werden Zeitreihen unterschiedlicher Skalierung besser vergleichbar und die Null-Problematik prozentualer Maße umgangen. Gleichzeitig hängt die Aussagekraft von der Stabilität des Skalierungsfaktors ab. Bei kurzen Zeitreihen, Strukturbrüchen oder ungeeigneter Saisondefinition kann die Skalierung selbst schwanken. In hierarchischen Datensätzen kann zudem die Wahl der Referenzdaten pro Ebene zu unterschiedlichen Verzerrungen führen [16].

EINORDNUNG IM KONTEXT DIESER ARBEIT. In dieser Arbeit werden 1-Step-Ahead-Direktprognose auf hierarchisch strukturierten Zeitreihen untersucht. Dafür werden Fehlermaße benötigt, die für Punktvorhersagen definiert sind, die als Trainingsziel in neuronalen Netzen praktikabel sind und die in der Literatur zur neuronalen Zeitreihenprognose etabliert sind [16,

35][8, S. 92][7, Kap. 5.1.4]. Vor diesem Hintergrund fällt die Wahl auf den mittleren quadratischen Fehler als Trainingsverlust und auf dessen Wurzel als Berichtsmetrik.

MITTLERER QUADRATISCHER FEHLER (MSE). Der MSE ist definiert als

$$\mathrm{MSE} = \frac{1}{n} \sum_{t=1}^{n} \left(y_t - \hat{y}_t\right)^2 \tag{2.4}$$

Terven et al. beschreiben den MSE als Standardmetrik, da durch das Quadrieren große Abweichungen stärker bestraft werden als kleinere. Das ist vorteilhaft, wenn große Fehler besonders kostspielig sind, erhöht jedoch die Empfindlichkeit gegenüber Ausreißern. Des Weiteren ermöglicht der MSE eine glatte und differenzierbare Gradientenbildung [35, S. 5ff.]. Als skalenabhängiges Maß ist der MSE zudem zwischen Zeitreihen unterschiedlicher Größenordnung nicht ohne weitere Normalisierung vergleichbar [35, S. 5ff.].

ROOT MEAN SQUARED ERROR (RMSE). Der RMSE ergibt sich als die Wurzel des MSE:

$$\mathrm{RMSE} = \sqrt{\frac{1}{n} \sum_{t=1}^{n} \left(y_t - \hat{y}_t\right)^2} \tag{2.5}$$

Der RMSE übernimmt die stärkere Gewichtung großer Abweichungen, führt den Fehler jedoch durch die Wurzel in die Originaleinheit zurück und unterstützt damit die fachliche Interpretation [16, 35]. Wie der MSE bleibt der RMSE skalenabhängig. Terven et al. diskutieren deshalb Normalisierungen, etwa über Streuungsmaße, wenn eine dimensionslose Vergleichbarkeit erforderlich ist [35, S. 17ff.].

BEGRÜNDUNG DER WAHL FÜR HIERARCHISCHE 1-STEP-AHEAD-DIREKTPROGNOSE. Der MSE wird als Verlustfunktion minimiert, da er eine effiziente und numerisch stabile Optimierung neuronaler Netze ermöglicht und in der einschlägigen Literatur als Standard etabliert ist [35][5, S. 129ff.][8, S. 97]. Der RMSE wird als Ergebniskennzahl berichtet, weil er auf der Originalskala interpretierbar ist und damit die Einordnung der Prognosegüte über Hierarchieebenen erleichtert [8, 16, 35]. Die Skalenabhängigkeit beider Maße wird akzeptiert, da die Arbeit keine Rangvergleiche über verschiedene Datensätze anstrebt. Der Analysefokus liegt auf der Veränderung der Prognosegüte zwischen Basismodell und Kohärenzregularisierung innerhalb desselben hierarchischen Systems [16].

KRITISCHE BETRACHTUNG. Durch die stärkere Gewichtung großer Fehler können Ausreißer, Messfehler oder Strukturbrüche das Training und die Bewertung dominieren [8, 16]. In hierarchischen Daten kann zusätzlich die Aggregation der Fehler über Zeitreihen dazu führen, dass große Reihen oder hohe Ebenen die Gesamtaussage prägen, sofern keine geeignete Normalisierung oder eine getrennte Auswertung nach Ebenen erfolgt. Daher müs-

sen die Daten normalisiert werden [8, 35]. Robustere Alternativen wie der Huber-Verlust [35, S. 12] sowie dimensionslose Kennzahlen wie MAPE wären denkbar, würden jedoch die Vergleichbarkeit dieser Arbeit mit Hengsens Ergebnissen und verwandter Literatur [15] einschränken. Daher werden MSE und RMSE bewusst eingesetzt, um den Effekt der Kohärenzregularisierung auf Kohärenz und Prognosegüte unter einem etablierten Standard isoliert untersuchen zu können [35, S. 5].

2.5 KOHÄRENZ

Wie in der Einleitung erwähnt, entsteht das Kohärenzproblem vor allem durch die getrennte Prognosebildung je Hierarchieknoten. Wenn jede Zeitreihe isoliert vorhergesagt wird, passen die Prognosen oft nicht mehr zu den bekannten Aggregationsbeziehungen der Hierarchie. Kohärenzbildung nutzt diese Information, um Prognosen über alle Ebenen so abzustimmen, dass sie konsistent sind und keine Widersprüche erzeugen. Dadurch werden die Ergebnisse leichter interpretierbar. Zusätzlich kann sich die Prognosegüte verbessern, weil das Modell an bekannte Zusammenhänge gebunden wird und Strukturinformationen zwischen Ebenen genutzt werden können [3].

2.5.1 *Grundgedanke der Kohärenzbildung*

Hierarchisch geordnete Zeitreihen besitzen, zusätzlich zu den in Abschnitt 2.2.2 erwähnten Eigenschaften, Aggregationsbeziehungen [3, S. 5]. Aggregationsbeziehungen definieren sich als semantische Verbindungen zwischen Zeitreihen. Grundlegend wird angenommen, dass die hierarchischen Zeitreihen klar trennbare, additive und einheitlich semantische[2] Aggregationsbeziehungen enthalten. So lässt sich aus den Aggregationsbeziehungen ein System linearer Gleichungen aufstellen, das gedanklich als gerichteter azyklischer Graph dargestellt werden kann, wie die folgende Abbildung zeigt:

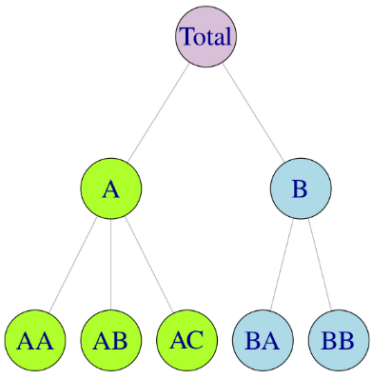

Abbildung 2.1: Eine generische Hierarchie, gebildet aus den disaggregierten Knoten AA, AB, AC, BA und BB. Aus [16, Kap. 11.1].

Ein System ist dann *kohärent*, wenn die Summe der Kindknoten dem Wert des Elternknotens entspricht. Alternativ kann die Kohärenzbedingung als Nullstellengleichung formuliert werden, wobei die Summe der Kindknoten abzüglich des Elternknotens gleich null sein muss [3, Kap. 2.2]. Inkohärenz bezeichnet die Abweichung vom kohärenten Zustand und kann als Distanzmaß zu diesem bemessen werden. Für diese Arbeit werden Kohärenzstörungen durch knotenweise Einzelprognosen eingeführt, was die realweltliche Tendenz zu isolierten Einzelprognosen abbildet [16, Kap. 11.2].

2 d. h. gleiche Einheiten und gleiche zeitliche Abhängigkeiten

2.5.2 *Notation und Summenmatrix-Entwicklung*

Die gedankliche Formulierung wird formalisiert, indem das System in Vektor- und Matrixform übergeführt wird. In Anlehnung an [16, Kap. 11] und [3, Kap. 2] wird folgende Notation verwendet:

- n bezeichnet die Anzahl aller Zeitreihen im System (alle Knoten über alle Ebenen).
- m bezeichnet die Anzahl der disaggregierten Zeitreihen auf den Blättern der Hierarchie (engl. *Bottom Level*), mit $m < n$.
- $n_a := n - m$ bezeichnet die Anzahl der aggregierten Zeitreihen (alle Nicht-Blatt-Knoten).
- $a_t \in \mathbb{R}^{n_a}$ bezeichnet den Vektor der aggregierten Beobachtungen zum Zeitpunkt t.
- $b_t \in \mathbb{R}^m$ bezeichnet den Vektor der Blattbeobachtungen zum Zeitpunkt t.
- $y_t \in \mathbb{R}^n$[3] bezeichnet den Gesamtvektor aller Knoten zum Zeitpunkt t, definiert als
$$y_t := \begin{bmatrix} a_t \\ b_t \end{bmatrix}$$
- I_m ist die $(m \times m)$-Einheitsmatrix.

Auf Basis der Hierarchiestruktur wird eine Aggregationsmatrix $A_{agg} \in \mathbb{R}^{n_a \times m}$ konstruiert. Jede Zeile von A_{agg} repräsentiert einen Elternknoten, jede Spalte ein Blatt. Ein Eintrag $A_{i,j}$ gibt an, ob (und ggf. mit welchem Gewicht) Blatt j in die Aggregation i eingeht. Im additiven Standardfall (Summenbildung) gilt typischerweise $A_{i,j} \in \{0, 1\}$:

$$A_{i,j} = \begin{cases} 1, & \text{falls Blatt } j \text{ Teil der Aggregation } i \text{ ist,} \\ 0, & \text{sonst.} \end{cases}$$

Allgemeiner können Gewichte $A_{i,j} \in \mathbb{R}$ verwendet werden, etwa für gewichtete Mittelwerte oder Indizes. Für diese Arbeit wird die Aggregation aus Modellierungsgründen als einfache additive Beziehung betrachtet. Damit lassen sich aggregierte Zeitreihen als lineare Abbildung der Blattwerte auf das Aggregat schreiben:

$$a_t = A_{agg} b_t. \tag{2.6}$$

3 Nicht zu verwechseln mit der Zeitreihe y_t aus Abschnitt 2.2.1.

SUMMENMATRIX S Um nicht nur die Aggregatwerte a_t, sondern den gesamten Systemzustand y_t aus den Blättern b_t zu erzeugen, wird die Summenmatrix (auch: strukturelle Matrix) $S \in \mathbb{R}^{n \times m}$ definiert als

$$S := \begin{bmatrix} A_{agg} \\ I_m \end{bmatrix}. \tag{2.7}$$

Damit gilt für das Gesamtsystem:

$$y_t = Sb_t. \tag{2.8}$$

KONZEPTIONELLE BEDEUTUNG UND KOHÄRENTER RAUM Die Summenmatrix S kodiert die vollständige Hierarchiestruktur als lineares System: Jede zulässige, d. h. kohärente Realisierung oder Prognose des Gesamtsystems liegt im von S aufgespannten *kohärenten Raum* $\mathcal{Y}$, formal definiert als:

$$\mathcal{Y} := \{\, y \in \mathbb{R}^n \mid \exists\, b \in \mathbb{R}^m \colon y = Sb \,\}$$

In anderen Worten: $\mathcal{Y}$ enthält genau jene Systemzustände, in denen alle Aggregationsregeln erfüllt sind. Sobald Blattwerte b_t gegeben sind, sind alle Werte auf höheren Aggregationsebenen durch S eindeutig bestimmt.

PROJEKTION BEI KOHÄRENZABWEICHUNG. Für Prognosen ist der Prognosevektor $\hat{y}_{t+h} \in \mathbb{R}^n$ im Allgemeinen nicht kohärent, da die Knoten typischerweise separat vorhergesagt werden. Das *zentrale Problem* besteht darin, einen kohärenten Systemzustand zu finden, der den Basisprognosen $\hat{y}_{t+h}$ möglichst nahekommt, d. h. die notwendige Korrektur minimiert.

2.5.3 *Prognoseabgleichsmethoden*

Forecast Reconciliation (dt. Prognoseabgleich) ist ein Sammelbegriff für Verfahren zur Kohärenzbildung auf prognostizierten Knoten einer hierarchisch strukturierten Zeitreihe [3]. Klassische Verfahren verfolgen das Ziel, die Kohärenz der prognostizierten Zeitreihen zu verbessern, ohne zwingend die zugrundeliegende Prognosemethode selbst zu ändern. Dies geschieht, indem inkohärente Basisprognosen nachgelagert in den kohärenten Raum überführt werden [3]. Historisch dominieren zwei Ansätze: strukturbasierte Aggregations- und Disaggregationsverfahren sowie projektions- bzw. regressionsbasierte Optimal-Combination-Verfahren.

STRUKTURBASIERTE ABGLEICHMETHODEN. Strukturbasierte Verfahren umfassen Aufwärts- (Bottom-up), Abwärts- (Top-down) und Middle-out-Methoden [15]. Bottom-up prognostiziert ausschließlich die disaggregierten Reihen und aggregiert anschließend mit S; es nutzt damit die maximale Granularität, kann jedoch unter starkem Rauschen der disaggregierten Zeitreihen leiden [15]. Top-down erstellt Prognosen auf hohen Aggregationsebenen und verteilt diese über Proportionsschlüssel auf nachgelager-

te Ebenen; es ist robust gegenüber Rauschen auf disaggregierter Ebene, kann jedoch feingranulare Dynamiken verfehlen [15]. Middle-out kombiniert beide Ideen, indem eine mittlere Ebene direkt prognostiziert und nach oben aggregiert bzw. nach unten disaggregiert wird, und vereint damit die jeweiligen Stärken und Schwächen beider Ansätze [16, Kap. 11.2].

Allen strukturbasierten Verfahren ist gemein, dass eine Ebene als Grundwahrheit angenommen wird und die Hierarchie auf dieser aufgebaut wird. Nachteilig ist dabei der Verlust von Informationen der übrigen Ebenen.

Darauf aufbauend wurden Optimal-Combination-Methoden entwickelt.

OPTIMAL COMBINATION / PROJEKTION (Aus [16, Kap. 11.5]) Zunächst werden die Zeitreihen auf allen Ebenen unabhängig voneinander prognostiziert und anschließend über eine lineare Projektion bzw. Regression in den kohärenten Raum transformiert. Ein grundlegender Ansatz ist die *Optimal Combination* via Regression nach der Methode der kleinsten Quadrate [15].

2.5.4 *Kohärenz durch kleinste Quadrate (OLS-Reconciliation)*

Kohärente Systemzustände lassen sich stets als $S\mathbf{b}$ (mit $\mathbf{b} \in \mathbb{R}^m$) schreiben, die durch die Summenmatrix S zu einem kohärenten Systemzustand führen, d. h. $\mathcal{Y} = \{S\mathbf{b}\}$ [3, 16]. Im Prognoseabgleich wird daher angenommen, dass die Basisprognose ein kohärenter Zustand plus Fehler ist:

$$\hat{\mathbf{y}}_{t+h} = S\mathbf{b} + \varepsilon,$$

und es wird die kohärente Prognose gesucht, die den quadratischen Abstand zu $\hat{\mathbf{y}}_{t+h}$ minimiert [15, 16]. Dazu wird der Vektor der disaggregierten Knoten $\hat{b}_{t+h}$ geschätzt, der $\hat{y}_{t+h}$ im Sinne der kleinsten Quadrate am besten erklärt:

$$\hat{b}_{t+h} = \mathbf{b}_{\text{opt}}, \qquad \mathbf{b}_{\text{opt}} \;=\; \arg\min_{\mathbf{b}\in\mathbb{R}^m} \frac{1}{n}\|\hat{\mathbf{y}}_{t+h} - S\mathbf{b}\|_2^2, \qquad \tilde{\mathbf{y}}_{t+h} = S\mathbf{b}_{\text{opt}}.$$

$\tilde{\mathbf{y}}_{t+h}$ ist damit *per Konstruktion* kohärent (Form $S\mathbf{b}$) und zugleich die im quadratischen Sinne nächstliegende kohärente Version der Basisprognose [3]. Äquivalent lässt sich dies als Projektion mit einem Projektor P auf den kohärenten Raum schreiben:

$$\tilde{y}_{t+h} \;=\; P\,\hat{y}_{t+h}, \qquad P \;:=\; SS^+,$$

wobei S^+ die Moore-Penrose-Pseudoinverse bezeichnet [24], da S im Allgemeinen nicht invertierbar ist. Die notwendige Korrektur lässt sich als

$$\boldsymbol{\Delta}_{t+h} := \hat{\mathbf{y}}_{t+h} - \tilde{\mathbf{y}}_{t+h}$$

interpretieren. Große Komponenten von $\boldsymbol{\Delta}_{t+h}$ bedeuten stärkere Verletzungen der Aggregationsbeziehungen.

2.5.5 *Bezug zur verwendeten Kohärenzmetrik*

Neben der Verwendung von P zur nachträglichen Erzeugung kohärenter Prognosen kann dieselbe Projektion als Referenz genutzt werden, um die *Inkohärenz* einer Basisprognose zu quantifizieren. Diese wird als mittlere Abweichung zwischen Basisprognose $\hat{y}_{t+h}$ und ihrer projizierten, kohärenten Version $P\hat{y}_{t+h}$ definiert:

$$\text{Kohrent}(\hat{y}_{t+h}) := \frac{1}{n} \left\| \hat{y}_{t+h} - P\hat{y}_{t+h} \right\|_1 .$$

GEWICHTETE OLS-VARIANTEN Darauf aufbauend wählen gewichtete Varianten von OLS die Projektion so, dass Knoten mit hoher Unsicherheit (hinsichtlich der Varianz von $y_{t+h} - \hat{y}_{t+h}$) weniger stark gewichtet werden [38, S. 7, 9, Theorem 1][37, S. 2]. Die praktischen Herausforderungen dieser Varianten bestehen in der Schwierigkeit der Schätzung der Fehlervarianz [38, S. 2, 7ff.][3, S. 435ff.][37, S. 2], im Speicherbedarf von $O(n^2)$ bei sehr großen Hierarchien [38, S. 23][3, S. 436ff.][37, S. 2] sowie darin, dass globale Optima der Kohärenz die Prognosegüte einzelner Basisprognosen verschlechtern können [37, S. 2]. Der derzeitige Standard unter den harten, gewichteten OLS-basierten Prognoseabgleichmethoden ist *Minimum Trace* (*MinT*) von Wickramasuriya et al. [38].

2.5.6 *ANN-basierte Ansätze*

Einen Paradigmenwechsel brachte die Kombination von Verlustfunktionen und Kohärenz: So wird Kohärenz nicht zwangsläufig nach der Prognose, sondern bereits während der Prognoseerstellung berücksichtigt [3, S. 438ff.]. Dies soll Informationsverluste und -verzerrungen harter Prognoseabgleichmethoden (vgl. Abschnitt 2.3.1) reduzieren und so Basisprognosegüte und Kohärenz verbessern [10, S. 1]. Durch die Integration von Aggregationsbeziehungen in typische Verlustfunktionen wird ein weiches Optimierungsziel gesetzt. Je nach Gewichtung kann Kohärenz verbessert werden, ohne die exakte Projektion in jedem Schritt zu erzwingen [3, 11]. Konzeptionell soll das ANN die Aggregationsregeln beachten, wobei leichte Abweichungen erlaubt sind [3, 11].

In der Literatur finden sich hierfür u. a. Bestrafungsterme, die differenzierbare Abgleichsschichten in ein Netzwerk einbetten sowie probabilistische Erweiterungen ermöglichen, um Kohärenz auch für Verteilungsparameter zu adressieren [3, S. 438].

ANGRENZENDE VERÖFFENTLICHUNGEN Die Ansätze unterscheiden sich vor allem darin, *wo* Kohärenz im Modell verankert wird: entweder an der Ausgabe, als eigene Abgleichsschicht oder auf Ebene von Verteilungsparametern.

Han et al. [10] koppeln Kohärenz an *quantilbasierte* (probabilistische) Prognosen. Das Modell erzeugt pro Knoten mehrere Quantile. Ein Kohärenz-

signal wird aus den vorhergesagten Knotenwerten abgeleitet, während zusätzliche Quantile die Unsicherheitsinformation liefern. Der Kohärenzterm ist damit direkt an die erzeugten Vorhersagen gekoppelt.

Rangapuram et al. [31] formulieren den Abgleich als *differenzierbare Rechenschicht* im Netzwerk. Die Projektion in den kohärenten Raum ist Teil der Vorwärtspassage, sodass das Modell end-to-end trainiert wird und Kohärenz im Rechenweg entsteht. Im Unterschied zu reinen Regularisierern wird hier eine explizite Abgleichsoperation in die Architektur eingebaut.

PROFHiT [18] adressiert Kohärenz im probabilistischen Setting über *Verteilungsparameter*. Neben der Prognosegüte wird ein Konsistenzterm optimiert, der die Hierarchie in den vorhergesagten Parametern (z. B. Lage und Streuung) einhält. Kohärenz ist damit ebenfalls ausgabenah, aber explizit verteilungsmotiviert.

UNTERSCHIEDE IN DER UMSETZUNG Die Ansätze unterscheiden sich darin, welchen zusätzlichen Mechanismus sie ins Training einbauen: Han et al. und PROFHiT benötigen pro Trainingsschritt die modellierten Ausgaben (Quantile bzw. Verteilungsparameter), um daraus den Konsistenzfehler zu berechnen. Rangapuram et al. ergänzen das Netz um eine eigene Abgleichsschicht, die in jeder Vorwärtspassage angewendet wird. Beide Ansätze binden den Kohärenzterm eng an die jeweils gesehenen Trainingsbeispiele.

WAS FÜHRT HENSGEN NEU EIN? Hensgen verschiebt die Kohärenz von den *Ausgaben* auf die *Netzwerkparameter* der letzten linearen Schicht [11]. Statt Kohärenz als Abweichung zwischen prognostizierten Knotenwerten zu bestrafen, gleicht NCL die finale Ausgabeschicht so an, dass sie strukturell zur Hierarchie passt. Konzeptionell ist das ein anderer Hebel: Kohärenz wird als Eigenschaft der Abbildung von latenter Repräsentation auf Knotenwerte modelliert, nicht als Zusatzbedingung auf einzelnen Vorhersagen. Hensgen begründet damit zwei erwartete Vorteile: geringere Kopplung an In-Sample-Outputs (potenziell weniger Overfitting auf Kohärenz) und robustere Kohärenz out-of-sample [11]. Zusätzlich nutzt Hensgen ein skalierbares Gewichtungsschema (α mit Inkrement-Logik), um den Zielkonflikt zwischen Genauigkeit und Kohärenz über das Training hinweg zu steuern [11]. Damit wird Kohärenz nicht starr gewichtet, sondern als steuerbares Optimierungsziel behandelt.

EINORDNUNG UND GRENZEN Im Vergleich zu Rangapuram et al. und PROFHiT liegt Hensgens Evaluation im Schwerpunkt auf *Punktprognosen* [11]. Methodisch ist NCL zudem an eine explizite *letzte lineare* Ausgabeschicht gebunden und wirkt unmittelbar nur auf deren Parameter. Die Methode ist daher nicht automatisch auf beliebige Ausgabemechanismen übertragbar.

Tabelle 2.1: Konzeptioneller Vergleich weicher Kohärenzansätze in neuronalen Modellen.

Arbeit	**Kohärenz verankert in**	**Prognoseart**	**Implementierungscharakter**
Han et al. [10]	Output (Quantile/Knotenwerte)	probabilistisch	Konsistenzterm aus vorhergesagten Outputs pro Schritt
Rangapuram et al. [31]	Abgleichsschicht im Netz	probabilistisch	Differenzierbare Projektion als Teil der Vorwärtspassage (end-to-end)
PROFHiT [18]	Output (Verteilungsparameter)	probabilistisch	Konsistenzterm auf Verteilungsparametern zusätzlich zur Likelihood
Hensgen [11]	Parameter der Ausgabeschicht	Punktprognose	Regularisierung der letzten linearen Schicht; Kohärenz als Netzwerkeigenschaft

2.6 NETWORK COHERENCY LOSS

2.6.1 *Grundgedanke*

Die Grundidee besteht darin, Kohärenz nicht über die fertigen Vorhersagen selbst zu erstellen, sondern über eine Regularisierung der *Netzwerkparameter* der letzten linearen Schicht eines ANN.

Angenommen, es handelt sich um ein

KERNIDEE: KOHÄRENZ ÜBER DIE LETZTE LINEARE SCHICHT REGULARISIEREN Formal definiert sich dieser Grundgedanke als Gewichtung der latenten Zustandsvektoren im Modell.

$$\hat{\mathbf{y}} = W_L \mathbf{a} + \mathbf{b}_L, \tag{2.9}$$

wobei $\mathbf{a}_t \in \mathbb{R}^{d_{\text{lat}}}$ die latente Repräsentation unmittelbar vor der letzten Schicht bezeichnet. Die Gewichtungsmatrix $W_L \in \mathbb{R}^{n \times d_{\text{lat}}}$ und der Vektor $\mathbf{b}_L \in \mathbb{R}^n$ sind die anpassbaren Parameter der letzten linearen Ausgabeschicht. Dabei entspricht n der Anzahl aller prognostizierten Zeitreihen (alle Knoten der Hierarchie). Jede Zeile von W_L enthält die Gewichte, mit denen die latenten Merkmale auf die jeweilige Zeitreihe abgebildet werden. W_i sind die Zeilen der Matrix, welche das Gewicht eines latenten Merkmals der i-ten Prognose darstellt, also den Index der Knoten. W_j sind die Spalten der Matrix, welche den Index der latenten Merkmals-dimension meint, eine Komponente von

a zum Zeitpunkt t , also den Index des Merkmals. $\mathbf{b}_L$ [4] Ausgleich, da sonst durch den Ursprung $a_t = 0$ und $\hat{y}_t = 0$ durchlaufen werden muss. So setzt sich die Prognose $\hat{y}_t + h$ zusammen.

Hensgen zeigt, dass sich der Kohärenzfehler der Vorhersage über eine obere Schranke ausdrücken lässt, die von der Norm der Aktivierungen $\|\mathbf{a}\|$ und von Abweichungen der Ausgabeschicht von der hierarchischen Struktur abhängt (vgl. Gleichungen (3.1) und (3.2) in [11]). Intuitiv gilt: Wenn die letzte Schicht so parametrisiert ist, dass sie mit der Summenmatrix S verträglich ist, wird es für das Modell leichter, kohärente Ausgaben zu erzeugen.

Aus dieser Schrankenbetrachtung definiert Hensgen die *Network Coherency Loss* als Regularisierung auf den Parametern der letzten Schicht:

$$\mathcal{L}_{\text{NCL}} \;=\; \|W_L - PW_L\|_F \;+\; \|\mathbf{b}_L - P\mathbf{b}_L\|_2, \tag{2.10}$$

[5] (vgl. Gleichung (3.3) in [11]). Da P eine Projektion in den Kohärenten Raum ist, kann dieser verwendet werden, um einen nicht-kohärenten Teil aus den ursprünglichen Gewichtungsmatrix W und dem Ausgleichswert b zu berechnen. Also den Fehler der zu minimieren ist, um eine kohärente Prognose zu erstellen. Die Werte von von W und b können jeweils dem Gradientenverfahren hinzugefügt werden A.4.

Damit wird nicht eine einzelne Vorhersage $\hat{\mathbf{y}}$ bestraft, sondern der *Mechanismus*, mit dem das Modell Hierarchieausgaben erzeugt. NCL wirkt somit wie ein struktureller Abgleich der Ausgabeschicht und macht Inkohärenz parametrisch unattraktiver. Da die Schranke mit $\|\mathbf{a}\|$ skaliert, verwendet Hensgen Batch-Normalization an [17] vor der letzten Schicht, um die Skalierung der Aktivierungen zu stabilisieren und das Zusammenwirken der Verlustterme zu erleichtern [11].

$$\mathcal{L} \;=\; \mathcal{L}_{\text{MSE}} \;+\; \cdot\mathcal{L}_{\text{NCL}}. \tag{2.11}$$

NCL kann einer anderen Verlustfunktion dazu addiert werden, da es keinen Ersatz für Prognosegüte ist. Daraus kann ein Zielkonflikt entstehen. Einerseits die Prognosegüte zu optimieren, anderseits die Kohärenz zu optimieren. Daher setzt Hensgen den Term α. Dieser soll eine Zielgewichtung vornehmen. Um so größer α ist, um so höher der Kohärenzdruck, um so schlechter die Prognosegüte und vice versa. Diese wird erweitert, dass sie sich nach Gradienten-updates inkrementiert. So definiert sich α als $\alpha = (Verlustfunktion \div NCL) \times \alpha_{increment}$ So entsteht die finale Gleichung:

$$\mathcal{L} \;=\; \mathcal{L}_{\text{MSE}} \;+\; \alpha \cdot \mathcal{L}_{\text{NCL}} \tag{2.12}$$

4 $\mathbf{b}_L$ ist ein Modellparameter und darf nicht mit den disaggregierten Werten $b_t \in \mathbb{R}^m$ aus Abschnitt 2.5.2 verwechselt werden.

5 Für $P := SS^+$ als Projektionsmatrix, in Hensgens Arbeit als A

2.6.2 *Experimentelle Methodik*

Hensgen evaluiert auf mehreren Datensätzen mit unterschiedlichen Hierarchietiefen und -größen. Verglichen wird ein Basismodell ohne Kohärenzmechanismus (nur MSE) mit einem Modell mit NCL sowie einem Modell mit NCL und zusätzlicher Projektion. Zur Parameterauswahl wird α (und ggf. α_i) über eine Validierungsmenge selektiert. Berichtet werden Mittelwert und Standardabweichung des

- **Tourism:** Basismodell RMSE $0.0148 \pm 2 \cdot 10^{-4}$, Projektion $0.0133 \pm 6 \cdot 10^{-4}$, bestes NCL-Modell (validierungsbasiert) mit $\alpha = 100, \alpha_i = 0.1$ RMSE $0.00955 \pm 2 \cdot 10^{-4}$ bei Kohärenz $\approx 9.49 \cdot 10^{-4}$ (Tabelle 4.3 in [11]).
- **Labour:** Basismodell RMSE 0.0737 ± 10^{-3}, Projektion $0.0724 \pm 4 \cdot 10^{-3}$, NCL (z. B. $\alpha_i = 0.1$) RMSE 0.0656 ± 10^{-5} (Tabelle 4.1 in [11]).
- **Traffic:** Basismodell RMSE $0.00384 \pm 3 \cdot 10^{-4}$, Projektion $0.0035 \pm 3 \cdot 10^{-4}$; validierungsbasiert selektiert Hensgen ein NCL-Modell mit hoher α-Initialisierung und Inkrement (z. B. $\alpha = 100, \alpha_i = 0.5$), das eine niedrigere Test-RMSE erreicht (Tabelle 4.2 in [11]).
- **M5:** Basismodell RMSE $0.00251 \pm 5 \cdot 10^{-5}$, Projektion $0.00248 \pm 5 \cdot 10^{-5}$. Sehr kleine α-Werte sind notwendig; ein "no-increment"-NCL mit $\alpha = 10^{-4}$ erreicht RMSE $0.00242 \pm 5 \cdot 10^{-6}$ (Tabelle 4.4 in [11]). Hensgen diskutiert, dass die geeignete Skalierung datensatzabhängig ist und bei großen Hierarchien (wie M5) hohe α-Werte zu instabilen oder verschlechterten Ergebnissen führen können und besser zu kleineren Datensätzen passen [11, S.16ff].
- **Private Dataset:** Die Projektion steigert Prognosegüte und Kohärenz gegenüber dem Basismodell (Tabelle 4.5 in [11]). Anzumerken ist, dass der Datensatz laut Hensgen [11, S. 16ff.] Fehlwerte enthält und ein längerer Prognosehorizont verwendet wurde, was die Prognosegüte beeinträchtigen kann. Dies deutet darauf hin, dass NCL die Prognose auch unter schlechteren Datenbedingungen und längeren Prognosehorizonten verbessern kann.

VERBESSERUNG DURCH **ncl!** IM VERGLEICH ZUM BASISMODELL Tabelle 2.2 fasst Hensgens (validierungsbasiert ausgewählte) NCL-Ergebnisse als prozentuale Verbesserung gegenüber dem Kontrollmodell ohne NCL zusammen. Der Varianzkoeffizient (VK)[6] gibt die Ergebnisrobustheit an und wird aus den von Hensgen angegebenen RMSE-Mittelwerten und Standardabweichungen berechnet. Die folgende Tabelle setzt sich aus den von Hensgen berichteten Tabellen zusammen [11, Tabellen 4.1–4.5].

6 Der Varianzkoeffizient ist definiert als $\mathrm{VK}_{\%} = 100 \cdot \frac{\sigma}{\mu}$ wobei σ die angegebene Standardabweichung und μ den RMSE-Mittelwert über fünf Läufe bezeichnet.

Tabelle 2.2: Relative, prozentuale Verbesserung der gemittelten Daten

Datensatz	ΔRMSE [%]	ΔKohärenz [%]	VK_{RMSE} Base [%]	VK_{RMSE} NCL [%]
Labour	10.99	3.03	1.36	0.02
Traffic	43.23	89.97	7.81	4.59
Tourism	35.47	86.44	1.35	2.09
M5	3.59	94.93	1.99	0.21
Private	1.47	66.62	1.22	1.11

Anzumerken ist die Robustheit der (VK) Ergebnisse. NCL scheint weniger Varianz in der Prognosegüte bei gleichzeitiger Verbesserung der Prognosegüte und Kohärenz zu verursachen. Die Test-RMSE steigt um mindestens 1.47%, die Kohärenz, um mindestens 3.03% unter schwereren Bedingungen.

2.6.3 *Kritische Einordnung*

Hensgens Ansatz ist methodisch attraktiv, da Kohärenz nicht als nachgelagerter "Korrekturschritt" behandelt wird, sondern als Trainingssignal, das die Modellparameter in Richtung kohärenter Ausgaben lenkt, dem Modell die "Regeln der Buchhaltung" beibringt. Gleichzeitig zeigen die Experimente mehrere Einschränkungen. Die starke Abhängigkeit von α (und α_i) erfordert mehr Forschung. Die Größenordnung von $\mathcal{L}_{\text{NCL}}$ skaliert mit der Ausgabe-Dimension, was insbesondere bei großen Hierarchien (wie in Hensgens M5-Datensatz[11]) zu sehr kleinen effektiven α-Werten führt. NCL wirkt ausschließlich auf die letzte lineare Schicht und setzt damit eine geeignete lineare Ausgabe-Parametrisierung voraus, wodurch unklar bleibt, wie universell die beobachteten Effekte über Modellklassen hinweg sind. Des weiteren wurde nur ein ANN Modell getestet, was die Wirkung von NCL auf andere ANN Modellen offen lässt.

2.6.4 *Implikationen für diese Arbeit*

Aus Hensgens Arbeit werden für die Methodik insbesondere die Verlustfunktion als $\mathcal{L}_{\text{MSE}}+\mathcal{L}_{\text{NCL}}$-Regularisierung, die validierungsbasierte Auswahl des Mischparameters sowie die Berichterstattung von Genauigkeits- und Kohärenzmetriken übernommen. Der zentrale offene Punkt ist die *Übertragbarkeit* der NCL-Idee auf andere Architekturen: Während Hensgen ein rekurrentes Modell nutzt, wird in dieser Arbeit ein Feed-Forward-basiertes Modell (N-BEATS) eingesetzt, um zu prüfen, ob die Kohärenz- und Genauigkeitsgewinne modellunabhängig sind oder von der ursprünglichen Netzwerkstruktur und Skalierung abhängen. Dies leistet zugleich einen Beitrag zur weiteren Erforschung der Grundidee der Kohärenzregularisierung und der Frage, ob eine Kohärenzverbesserung systematisch auch die Prognosegüte verbessert. Ergänzend wird die Robustheit der Ergebnisse über mehrere Trainingsläufe betrachtet [11].

3 METHODIK

Die Methodik basiert auf dem Versuchsaufbau von Hensgen [11] und folgt in der Prozessgestaltung den Empfehlungen von Hyndman [16, Kap. 1.6], Hewamalage et al. [12, Kap. 4.3–5], Haben [8, S. 229, 283] und Hensgen [11]. Zunächst wird die Testumgebung beschrieben, anschließend die verwendeten Daten und die enthaltene Hierarchie sowie deren Vorverarbeitung. Darauf folgt die Beschreibung der Modellarchitektur und Implementierung von N-BEATS, der Implementierung von NCL, des Balancierungsparameters α und der Kohärenzmetrik, sowie eine Übersicht der Trainingsparameter. Abschließend werden die wesentlichen Limitierungen dargelegt und zur Ergebnisdarstellung übergeleitet. Die Änderungen:

3.1 ERWARTUNGEN UND EINGRENZUNG

Hensgens NCL Ergebnisse zeigen eine generelle Verbesserung der Kohärenz als auch der Prognosegüte auf seinen Testdatensätzen. Hensgen nimmt auch an, dass diese Verbesserungen Modellagnostisch sind. Daraus folgt die Arbeitsannahme, dass für ein anderes ANN basiertes Modell ähnliche Verbesserungen eintreten würden. Konkret werden folgende Hypothesen getestet:

- H1: NCL Verbessert die Prognosekohärenz, um mindestens 86.44% auf Feed-Forward-basierten ANN gegen ein Basismodell auf australischen Tourismusdaten.
- H2: NCL Verbessert die Prognosegüte hinsichtlich der RMSE, um mindestens 35.47% auf Feed-Forward-basierten ANN gegen ein Basismodell auf australischen Tourismusdaten.
- H3: NCL reduziert möglicherweise die Prognosegütevarianz.

Daher wird auf Hensgens Methodik-Basis ein weiteres ANN basiertes Modell auf dem von Hensgen verwendeten Australian-Tourism Datensatz getestet.

3.2 TESTUMGEBUNG UND VERWENDETE SOFTWARE

Die Experimente werden auf Windows 11 in einer containerisierten Python-Umgebung ausgeführt, um eine reproduzierbare und öffentlich einsehbare Durchführung der Trainingsläufe sicherzustellen. Als Software werden ausschließlich die für Datenimport, Modelltraining und Visualisierung benötigten Bibliotheken herangezogen: **PyTorch 2.4.0** für Modellimplementierung, Training und GPU-Beschleunigung (inkl. `torch.nn` sowie `torch.utils.data` für `Dataset/DataLoader`); **NumPy 1.26.4** für numerische Operationen und Matrixkonstruktionen; **Pandas 2.2.2** für Datenimport und -aufbereitung; **Matplotlib 3.8.4** für die Ergebnisvisualisierung (Trainingskurven und Metrikgraphen). Die Ausführung erfolgt abhängig von der Verfügbarkeit einer CUDA-fähigen GPU wahlweise auf `cuda` oder `cpu`; die Geräteauswahl wird zur Laufzeit automatisch getroffen. Zur Verbesserung der Reproduzierbarkeit werden nicht-deterministische NVIDIA-cuDNN-Optimierungen deaktiviert.[1] Weitere Details sind im GitHub-Repository unter Anhang A.1 zu finden.

3.3 DATENSATZ

Hensgen nutzt den in der Literatur etablierten [3, S. 445] Australian-Tourism-Datensatz aus der Veröffentlichung von Athanasopoulos et al. [2]. Dieser wird erweitert. Anstelle von 255 werden 555 Zeitreihen untersucht. Die zusätzlichen Zeitreihen werden aus den Basisdaten erstellt, durch Kreuzprodukte und als Zwischen-Hierarchien eingefügt [1, S. 9ff.]. Dieser Datensatz ermöglicht Vergleiche nicht nur mit Hensgen, sondern auch mit der breiteren Literatur zur Evaluation von Prognoseabgleichmethoden und erhöht den Komplexitätsgrad der Daten nach 2.2.2. Des Weiteren sind Zerlegungen der Daten öffentlich verfügbar wie (vgl. Hyndman [16, Kap. 4.5]).

Es wird angenommen, dass der Datensatz ein nicht-triviales Prognoseproblem im Sinne von Abschnitt 2.2.2 darstellt. Eine formale Überprüfung war aus Umgangsgründen nicht möglich. Die Annahme stützt sich darauf, dass frühere Forschung den Datensatz aufgrund seiner Komplexität, Nichtlinearität und handhabbaren Größe wiederholt als Benchmark gewählt hat.

3.3.1 *Datenerläuterung*

Die in dieser Arbeit verwendete CSV-Datei [32] enthält:

- **Frequenz**: Monatlich die Reisezwecke von Touristen, aufgeteilt nach Land, Bundesland, Region.
- **Zeitraum**: Von Januar 1998 bis Dezember 2016 mit (T=228) Zeitpunkte
- **Blätter**: Mit 304 disaggregierte Zeitreihen.

1 Verwendete Hardware: Intel Core i5-8300H, NVIDIA GeForce GTX 1050 Ti, 16 GB RAM.

Jeder Blattknoten ist durch einen dreibuchstabigen Regionscode und einen Reisezweck-Suffix codiert in Form von:

`<Region(3 Buchstaben)><Purpose>` mit `Purpose` $\in \{$`Hol, Vis, Bus, Oth`$\}$.

Beispielsweise bezeichnet `AAAHol` den Zweck `Hol` (Holiday) in Region `AAA`.

3.3.2 *Hierarchiekonstruktion (State, Zone, Region, Purpose)*

Da die Datei die Blätter (Region$\times$Purpose) liefert, werden alle höheren Ebenen durch Summieren konstruiert. Die Hierarchieebenen folgen der in den Regions-Codes enthaltenen geografischen Struktur:

- **State**: Erstes Zeichen des Regionscodes
- **Zone**: Erste zwei Zeichen
- **Region**: Drei Zeichen (Blatt-Geografie)
- **Purpose**: `Hol, Vis, Bus, Oth`

Die in dieser Arbeit verwendete Ausgaberepräsentation umfasst acht Ebenen nach Ashouri et al. [1]:

1. Total (1)
2. Purpose (4)
3. State (7)
4. State$\times$Purpose (28)
5. Zone (27)
6. Zone$\times$Purpose (108)
7. Region (76)
8. Region$\times$Purpose (Bottom-Level: 304)

Damit ergibt sich eine Systemdimension $\mathbb{R}^n$ von

$$n = 1 + 4 + 7 + 28 + 27 + 108 + 76 + 304 = 555.$$

3.3.3 *Datenvorverarbeitung*

Die Daten müssen vorverarbeitet werden, da sonst die von Hensgen entwickelte Alphaskalierung zu stark verzerrt wird und MSE-Skalierung überhand nimmt (siehe A.5) sowie die in 2.4.2 besprochenen Probleme reduziert werden.

NUMERISCHE DATENVORVERARBEITUNG Die Daten liegen bereits in einem bereinigten Zustand vor. Sporadische Fehlwerte und Typinkonsistenzen wurden vorab behandelt. Fehlende Monatswerte werden mittels Vorwärtsfüllung ergänzt, um eine eindeutige zeitliche Zuordnung sicherzustellen. Damit wird die zeitliche Kopplung der Monate zu den Regionen und dem Reisezweck (*Purpose*) hergestellt.

Alle Fließkommawerte werden als `float32` typisiert. Dies reduziert nach Jan Sellner den Speicherverbrauch und beschleunigt das Modelltraining [33]. Eine `float16`-Typisierung wurde aufgrund beobachteter Leistungsverschlechterungen verworfen; `float64` entfällt, da der Industriestandard moderner Grafikkarten auf 32-Bit-Arithmetik ausgelegt ist.

SUMMENMATRIXBERECHNUNG Nach 2.5.2 wird die Summenmatrix aus den Blätterknoten der Daten erstellt. Diese bildet sich aus $b_t \in \mathbb{R}^{304}$ Blatt-Zeitreihen zu t_0 bis t_{227}. Alle Hierarchieknoten werden in $y_t \in \mathbb{R}^{555}$ gesammelt. Die Aggregationsbeziehungen werden durch die Summenmatrix $S \in \mathbb{R}^{555\times304}$ beschrieben. Nach der Erstellung der Summenmatrix werden die jeweiligen Zeitreihen normalisiert. Dieser Schritt erfolgt nach der Bildung der Summenmatrix, damit diese durch die Normalisierung keine verzerrten Strukturen aufnimmt.

DIE NORMALISIERUNG Nach dem Aufbau der Summenmatrix werden alle Zeitreihen mittels Min-Max-Normalisierung [8, S.75] skaliert. Auf jede einzelne Zeitreihe y_t wird

$$y_t^{(\text{skaliert})} = \frac{y_t - y_{\min}}{y_{\max} - y_{\min} + \varepsilon} \tag{3.1}$$

(vgl. [8, S.74ff]) angewendet, sodass $y_t^{(\text{skaliert})} \in [0,1]$ normalisiert wird. [2] . Die Normalisierung verfolgt in diesem Versuchsaufbau drei Ziele:

1. **Stabilisierung der Gradientendynamik** Bei großen Zahlenwerten erzeugt der MSE skalierte Gradienten, die zu überproportionalen Parameterupdates und damit zu instabilem Training führen können. Eine Normalisierung kontrolliert diese Skalierung und begünstigt die Konvergenz (vgl. Anhang A.3).

2. **Robustheit gegenüber ausreißenden Reihen und Hierarchieebenen** Die Tourism-Hierarchie umfasst Zeitreihen mit stark unterschiedlichen Größenordnungen. Eine Min-Max-Normalisierung reduziert die Dominanz großskaliger Reihen und stellt sicher, dass kleinere Reihen beim Training nicht ignoriert werden [8]. Dadurch wird das Lernsignal gleichmäßiger über die Hierarchie verteilt und die Modellanpassung weniger von einzelnen hochskaligen Aggregaten abhängig (vgl. Abschnitt 2.4.2).

2 ε dient der numerischen Stabilität für den Spezialfall $y_{\max} = y_{\min}$.

3. **Vergleichbarkeit der Verlustterme für die α-Skalierung** Im NCL-Modell setzt sich der Trainingsverlust aus MSE und Kohärenzregularisierungsterm zusammen. Ohne Normalisierung kann der MSE die Optimierung dominieren, sodass die Regularisierung wirkungslos bleibt oder zu extremen α-Werten führt. Eine Min-Max-Normalisierung reduziert diese Skalenverzerrung, sodass MSE und NCL in vergleichbaren Größenordnungen liegen und α über die Epochen kontrollierbarer bleibt.

Es existieren weitere Normalisierungsverfahren (siehe. [20], [8, S.72]), aber Kim et al. [20] berichten gemischte Effekte in einer breiten Anwendung auf verschiedene Rekurrente ANN, welche eine definitive Leistungsüberlegenheit, je Verfahren und Testumgebung, ausschließt. [20]. Vorsichtig wird angenommen, dass diese Ergebnisse auch übertragbar auf Feed-Forward basierte ANN sind. Des weiteren ermöglicht Min-Max Normalisierung die Rückskalierung in die ursprüngliche Einheit, welche nicht möglich wäre bei anderen Normalisierungsverfahren.

DATENAUFTEILUNG Die normalisierten Daten werden anschließend aufgetrennt in Trainingsdaten, Validierungsdaten und Testdaten. Diese Trennung wird abgeleitet aus Haben et al.[8, S.113], Hewamalage et al.[12, S.401] und Hensgen [11, S.25]. Deren Details sind in 3.1 zu finden.

ZUSAMMENGEFASST Die Daten werden durch das Weiterführen der zeitlichen Daten aufgewertet und auf float32 typisiert, um schnelles und stabileres Modelltraining zu gewährleisten. Die Hierarchie und Summenmatrix werden aus den Blätter-Zeitreihen erstellt. Im Anschluss werden alle Zeitreihen normalisiert und in Trainings-, Validierungs- und Testdaten aufgetrennt.

3.4 TRAININGSMODELL (N-BEATS)

In dieser Arbeit wird ein neuronales Basisexpansionsmodell für Zeitreihen (eng. *Neural Basis Expansion Analysis for Time Series*, N-BEATS) verwendet [27]. N-BEATS ist ein *Feed-Forward*- Prognosemodell, das explizit für Punktprognosen auf Zeitreihen entwickelt wurde und ohne handgefertigte Merkmalskonstruktion auf Roh-Zeitfenstern arbeiten könnte [27, S.2]. N-BEATS findet breite Anwendung in diversen Prognosebereichen, wie z./B. zur Prognose der Strom-Nachfrage [19]. Des weiteren verwendete die Originalveröffentlichung von N-BEATS den australischen Tourismus-Datensatz zur Leistungsmessung [27] und wurde zum testen einer Kohärenzverlustfunktionen schon verwendet [3, S.438].

3.4.1 *Architekturbeschreibung*

Der zentrale Architekturgedanke von N-BEATS ist die Zerlegung des Lernproblems in MLP-Blöcke. Jeder Block enthält zwei Ausgabeköpfe: einen Rückschauanteil (engl. *backcast*) und einen Prognoseanteil (engl. *forecast*). Der Rückschauanteil wird als Restsignal (engl. *residual*) an den nächsten Block weitergegeben; er repräsentiert den noch nicht erklärten Anteil der Eingabe. Der Prognoseanteil wird als Blockausgabe akkumuliert. Ein Stack fasst mehrere solcher Blöcke zusammen und aggregiert deren Prognoseanteile, gibt jedoch das Restsignal an den nächsten Stack weiter. Abschließend werden die Prognoseanteile aller Stacks summiert, um eine einheitliche Gesamtprognose zu erzeugen[27].

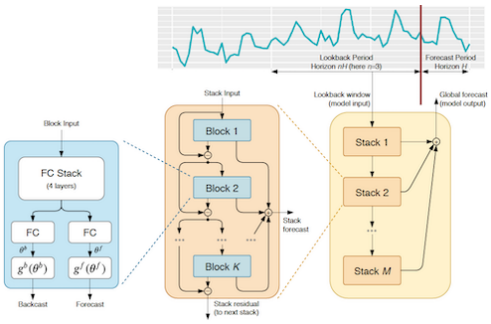

Abbildung 3.1: N-BEATS Blöcke, Blockinhalt und Aufteilung aus [27, S.3]

3.4.2 *Notation*

BEDEUTUNG DER WICHTIGSTEN SYMBOLE

- y_t: Beobachteter Zeitreihenwert zum Zeitpunkt t.
- L: Anzahl vergangener Werte, die das Modell als Eingang sieht.
- $H, h \in H$: Anzahl zukünftiger Werte, die das Modell prognostiziert.
- $x_t \in \mathbb{R}^L$: Eingabefenster (die letzten L Beobachtungen).

- K: Anzahl der Blöcke.
- $\hat{a}$: ist der interne Zustand einer Prognose im ANN. Dafür gilt $\hat{a} \in \mathbb{R}^{B \times d_{intern}}$.
- d_{latent}: Eingabedimension nach Durchlauf eines MLP-Blocks.

3.4.3 *Modellumsetzung und Pipeline*

Die Modellpipeline orientiert sich an Hensgens Vorgehen und ist im Code als N-BEATS-ähnlicher Stack implementiert. Ein Trainingsbeispiel besteht aus einem Stapel (Batch) von Eingabefenstern $X_t \in \mathbb{R}^{B \times L \times n}$, wobei B die Stapelgröße, L die Rückschaufensterlänge und n die Anzahl der Hierarchieknoten ist. Im vorliegenden Aufbau wird ein *1-Step-Ahead-Direktprognose* Ziel verwendet ($h = 1$).

PIPELINEÜBERSICHT Die Verarbeitungspipeline lässt sich kompakt wie folgt schreiben:

$$X_t \rightarrow \text{Flatten} \rightarrow \text{Encode} \rightarrow \text{N-BEATS} \rightarrow \hat{a} \rightarrow \text{Batch-Norm.}$$

$$\rightarrow \text{Linearkopf} \rightarrow \hat{y}_t \text{ (ähnlich Hengsen [11, S.15])}$$

FLATTEN UND ENCODE Zunächst wird jedes Fenster $x_t \in X_t$ zu einem Vektor umgeschrieben mittels $x.rehape(B, -1)$ [30, torch.reshape]. Dies reduziert die Eingabedimension auf einen eindimensionalen Vektor und erleichtert die Datenweitergabe.

Da $L \cdot n$ in hierarchischen Datensätzen sehr groß sein kann, wird ein Kodierer (Encoder) verwendet, der die Eingabe in einen kompakteren Merkmalsraum projiziert durch [30, torch.nn.Module]. Dadurch wird eine konstante interne Dimension erstellt, wodurch sich der Rechenaufwand steuern lässt. Dieser wurde auf $d_{encoder} = 1024$ gesetzt. Die Wahl wurde heuristisch getroffen nach Ablation über $d \in 256, 512, 1024, 2048$.

N-BEATS Das N-BEATS-Modell wurde vereinfacht aus MLP-Blöcken aufgebaut, welcher von der N-BEATS-Implementierung von Oreshkin et al. abweicht. Da dieser keinen Stack zur Trend-/Saisonalität enthält [27]. Das Modell ist wie folgt aufgebaut: Es gibt einen Stack, der aus jeweils $K = 6$ Blöcken besteht. Jeder Block ist ein MLP mit jeweils 4 linearen Schichten, welche jeweils 512 Neuronen Breit sind [3] [7, Kap.6, Kap.11]. Diese sind wie in der Architektur beschrieben Verbunden 3.4.1 und erstellen hierarchisch Systemglobale Prognosen für $h = 1$ mit einem $L = 50$.

3 Als Aktivierungsfunktion dient Rectified Linear Unit (ReLU). Die Funktion ist eine lineare Funktion, die nur bei positiven Werten ansteigt. Laut Goodfellow et al. (Kap.6, 11) eine standard funktion für ANN

BATCH-NORMALISIERUNG DER PROGNOSE-SUMME Auf die Summe von $\hat{a}$ wird Batch-Normalisierung angewendet:

$$\tilde{a} := \mathrm{BN}(\hat{a}) \in \mathbb{R}^{B \times d}.$$

Dies stabilisiert die Skala der Prognosewerte-Repräsentation, insbesondere weil $\hat{a}$ als Summe mehrerer Blockbeiträge entsteht und ansonsten einzelne Blöcke die Aktivierungsskala dominieren könnten[7, S.313ff]. Des Weiteren führt Batch-Normalisierung zu schnellerer Konvergenz und reduziert die Sensitivität gegenüber initialisierten Hyperparameter [7, S.133ff]. Aus diesen Gründen nutzt Hensgen Batch Normalisierung in seiner Arbeit [11, S.14] Die Batch-Normalisierten Daten werden dem Linearkopf weitergegeben.

LINEARKOPF ZUR AUSGABE Die endgültige Prognose für alle n Hierarchiereihen wird durch einen linearen Kopf erzeugt in dem Hensgens NCL Implementiert wird. Dieser nimmt die Summe von $\hat{a}$ zum Zeitpunkt t, gewichtet diese mit $W \in \mathbb{R}^{n \times d_{latent}}$ und addiert b, welcher durch Gradientenupdates verbessert wird, um $b - Ab$, also die Entfernung zum kohärenten Raum minimiert.

$$\hat{y}_t := \tilde{a}W^\top + b^\top \in \mathbb{R}^{B \times n}, \qquad W \in \mathbb{R}^{n \times d_{\mathrm{lat}}},\ b \in \mathbb{R}^n.$$

Der Linearkopf gibt anschließend die finale Prognose $\hat{y}_t$ aus.

3.4.4 *Trainingsparameter Übersicht*

Optimiert wird mit Adam[21, 29], [7, S.305] ($\eta = 0.005$, Weight Decay $= 0$). Es werden 1000 Epochen trainiert, wobei jede Epoche genau 25 Batch-Updates umfasst (d. h. 25 Gradientenoptimierungsschritte pro Epoche [4]). Die Batchgröße wird auf 32 gesetzt (notwendige Spezifikation, da 25 Batches pro Epoche die Batchgröße nicht eindeutig festlegt). Pro Modellvariante werden fünf Testläufe mit unterschiedlichen Zufallsseeds durchgeführt. Als Kontrollpunkt wird pro Testlauf das Modell mit der niedrigsten Validierungs-RMSE gewählt. Die Hyperparameter richten sich nach Hensgen [11] und Hewamalage et al.[12, S.405ff].

Die folgende Tabelle fasst die verwendeten Hyperparameter zusammen.

4 Batches definiert nach [8, S.176],[7, S.275]

Tabelle 3.1: Hyperparameter und Protokollparameter der Experimente.

Kategorie	**Wert**
Lookback L	50
Horizon H	1
Benchmark-Block	letzte 223 Monate (Jun 1998–Dez 2016)
Split	150 / 25 / 48 (Train/Val/Test)
Epochs	1000
Updates pro Epoche	25 Mini-Batches
Batchgröße	32
Optimizer	Adam
Basis-LR η	0.005
Weight Decay	0.0
Läufe	5 über Seeds 1337 bis 1341
Encoder-Dim	1024
N-BEATS Blöcke K	6
MLP-Tiefe je Block	4
MLP-Breite je Layer	512
Latent-Dim d	512
BatchNorm auf Latent	ja
Dropout	0.0
NCL: α_s Start	100.0
NCL: α_s Inkrement	0.1 alle 100 Epochen
NCL: Fenster p	100
NCL: Lernrate-Skalierung	$\eta' = \eta/(1 + \log(1 + \alpha/2))$ nach Hensgen [11]

3.4.5 *Ausgabelogik*

Nach dem Modelldurchlauf werden die normalisierten Daten rückskaliert[5], um einen Vergleich sowohl zwischen den Modellen als auch mit Hengsens Ergebnissen zu ermöglichen. Die rückskalierten Ergebnisse werden als RMSE berichtet. Des Weiteren werden Graphen zur Kohärenzentwicklung pro Epoche sowie zur α-Trainingsdynamik über die Epochen ausgegeben.

5 Die Rückskalierung erfolgt gemäß $x_{\text{original}} = x_{\text{norm}} \cdot (x_{\max} - x_{\min}) + x_{\min}$ (vgl. [8, S. 74]).

4 ERGEBNISSE

In diesem Kapitel werden die experimentellen Ergebnisse zur Evaluation der *Network Coherency Loss* (NCL) nach Hensgen berichtet. Verglichen wird ein Basismodell ohne NCL mit einem Modell mit NCL; als Architektur dient N-BEATS, als Datengrundlage der Australian-Tourism-Datensatz nach Ashouri et al. [1] und Athanasopoulos et al. [2]. Als Evaluationsmaße dienen RMSE und Kohärenz (Kohä.). Alle Ergebnisse gelten für $\alpha = 100$ und $\alpha_i = 0.1$.

Das Kapitel gliedert sich in die Präsentation der Hauptresultate, einen tieferen Einblick in die Ergebnisrobustheit sowie die Analyse der Trainingsdynamik. Abschließend werden die Ergebnisse mit Hengsens Arbeit verglichen, sowie die Limitierungen der Arbeit diskutiert.

4.0.1 *Hauptresultate*

Tabelle 4.1 fasst die Kennzahlen als Mittelwert ± Standardabweichung über fünf Seeds zusammen. NCL verbessert sowohl die Kohärenz als auch die Prognosegüte. Im Mittel sinkt der Test-RMSE von 373.40 auf 246.44 (relative Verbesserung: 34.0 %), die Kohärenz von 1.747 auf 0.133 (relative Verbesserung: 92.4 %) und der Beste-Validierungs-RMSE (B.Val-RMSE von 325.54 auf 154.45 (relative Verbesserung: 52.6 %). Gleichzeitig sinken die Standardabweichungen aller Metriken. Die Streuung des Test-RMSE reduziert sich relativ um 20.2 %, die der Test-Kohärenz um 2.6 %; ein ähnliches Bild zeigt sich beim Train-MSE und beim Validierungs-RMSE.

Tabelle 4.1: Ergebnis-Zusammenfassung (Mittelwert ± Standardabweichung über Seeds, Originaleinheiten). Δ% ist relative Verbesserung von NCL gegenüber Basismodell.

Modell	Test RMSE	Test Kohärenz	B.Val- RMSE
Basismodell	373.40 ± 114.14	1.747 ± 0.383	325.54 ± 102.92
ncl	246.44 ± 6.29	0.133 ± 0.034	154.45 ± 3.60
Δ% (B → NCL)	34.0% ± 20.2%	92.4% ± 2.6%	52.6% ± 15.0%

Ergänzend zeigt Tabelle 4.2 die Einzelläufe pro Seed. In 4 von 5 Fällen verbessert NCL den Test-RMSE um mindestens 7.2 %; Seed 1340 stellt mit −7.44 % den einzigen negativen Ausreißer dar. Die Test-Kohärenz verbessert sich in allen fünf Fällen um mindestens 88.89 %. Der Best-Val-RMSE verbessert sich um mindestens 25.71 %.

Tabelle 4.2: Einzelruns pro Seed (Originaleinheiten). Δ in % relativ zum Basismodell: $\Delta[\%] = \frac{B-NCL}{B} \cdot 100$ (positiv = NCL besser).

Seed	$RMSE_B$	$RMSE_{NCL}$	ΔRMSE [%]	$Kohä._B$	$Kohä._{NCL}$	Δ Kohä.[%]	B.Val-$RMSE_{B/NCL}$	ΔB.Val-RMSE [%]
1337	259.50	240.83	7.20	1.968	0.177	90.99	222.10 / 160.70	27.64
1338	467.26	242.61	48.08	1.390	0.154	88.89	409.39 / 153.04	62.62
1339	465.05	245.88	47.13	2.102	0.090	95.70	409.36 / 151.81	62.92
1340	239.20	256.99	-7.44	1.277	0.116	90.93	205.26 / 152.49	25.71
1341	435.98	245.88	43.60	1.999	0.129	93.55	381.59 / 154.21	59.59

4.0.2 *RMSE-Standardabweichung über Seeds*

Zur Robustheitsbewertung wird die Streuung des Test-RMSE über fünf Seeds betrachtet. Das Basismodell weist eine Standardabweichung von $\sigma \approx 114.14$ auf, während NCL mit $\sigma \approx 6$ etwa um den Faktor 100 stabiler ist (vgl. Tabelle 4.1). Dies entspricht einer relativen Reduktion der Streuung von 20.2 %. Ein ähnliches Bild zeigt sich beim B.Val-RMSE. Dies deutet darauf hin, dass die Regularisierung RMSE und Kohärenz nicht zufällig verbessert, sondern die Ergebnisse gegenüber einer reinen MSE-Verlustfunktion robuster macht.

4.0.3 *Trainingsdynamik*

Zur Interpretation der Resultate bietet ein qualitativer Überblick über die Trainingsdynamik zusätzliche Erkenntnisse. Exemplarisch wird Seed 1337 hinsichtlich des Validierungs-RMSE und des Kohärenzverhaltens für Basis- und NCL-Modell verglichen.

KOHÄRENZ ÜBER EPOCHEN Der folgende Graph zeigt den Kohärenzverlauf des Basismodells über 1000 Epochen. Auffällig sind zwei Sprünge zwischen Epoche 0 und Epoche 100: Die Kohärenz verschlechtert sich zunächst stark - von ca. 4000 auf ca. 12000 - und verbessert sich anschließend abrupt. Bis ca. Epoche 100 pendelt sich der Wert ein.

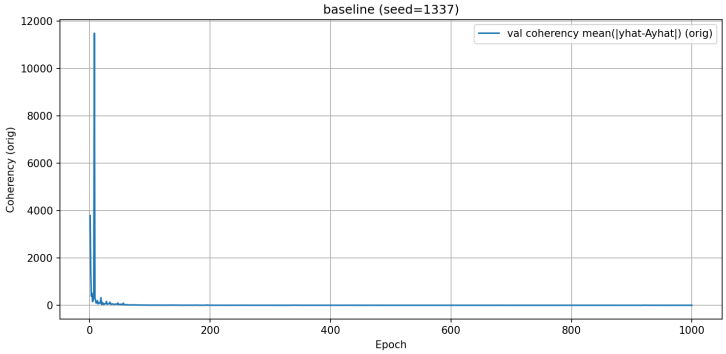

Abbildung 4.1: Exemplarischer Kohärenzverlauf des Basismodells

Dagegen steht die Kohärenz des NCL-Modells. Hier ist ein sofortiger Abfall von ca. 550 auf nahe 0 zu sehen. Dies ist zu erwarten, da eine schon Kohärente-Basisprognose aus dem Endkopf ausgegeben wird. Der Graph bleibt bis zum Ende der Validierung flach, was bedeutet, dass während der Validierung keine Fehler auftreten.

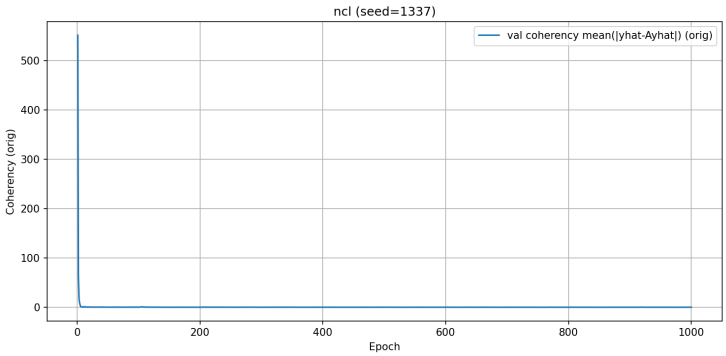

Abbildung 4.2: Exemplarischer Kohärenzverlauf des NCL-Modells

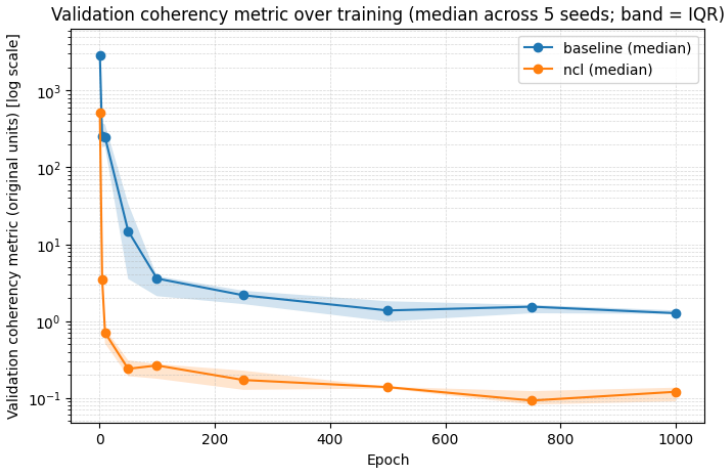

Abbildung 4.3: Basis und NCL Validierungskohärenz logarithmisch dargestellt

Im Vergleich der Validierungs-Kohärenz auf einer logarithmischen Skala wird das Kohärenzverlustverhalten deutlicher. NCL minimiert den Kohärenzfehler schneller und stärker als das Basismodell. Dies war aus Hensgens Ergebnissen zu erwarten.

VALIDIERUNGS-RMSE ÜBER EPOCHEN Auffällig ist die Wiederkehr der Spitzen aus Abbildung 4.1 zu denselben Epochen. Aus beiden Graphen lässt sich schließen, dass eine bessere MSE-Prognose tendenziell mit einer kohärenteren Prognose einhergeht.

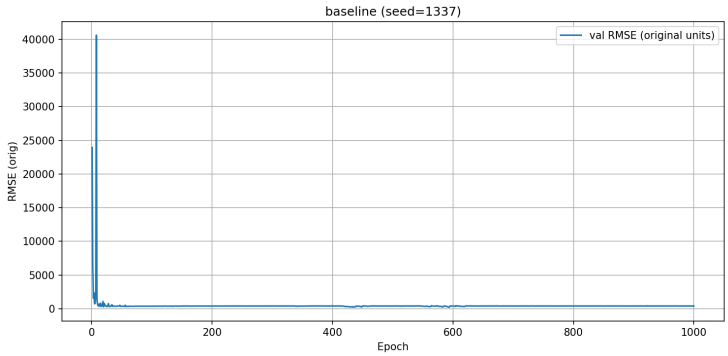

Abbildung 4.4: Validierungs-RMSE des Basismodells über Epochen

Im Vergleich der RMSE-Validierung gilt die gleiche Überlegung von 4.4 nicht ganz für das NCL-Modell. Dieses fällt sofort ab, steigt leicht wieder an und setzt dann eine Zackenbewegung fort. Anzumerken ist, dass aufgrund der niedrigeren Start-RMSE deutlich mehr Zackenbewegungen zu sehen sind.

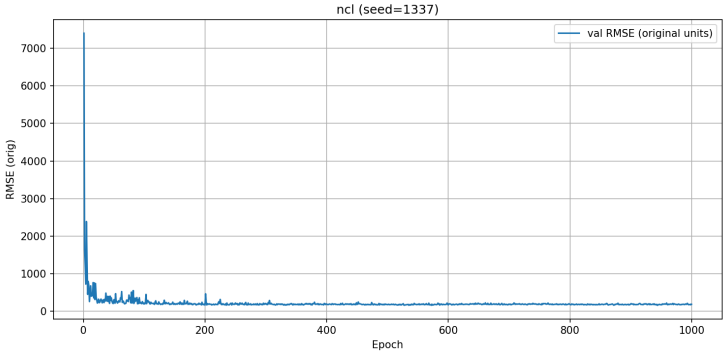

Abbildung 4.5: Validierungs-RMSE des NCL-Modells über Epochen

Ein Vergleich der Validierungs-RMSE auf einer logarithmischen Skala verdeutlicht das RMSE-Verhalten. NCL minimiert den RMSE stärker als das Basismodell und tut dies auch nach Epoche 100, wo das Basismodell stagniert.

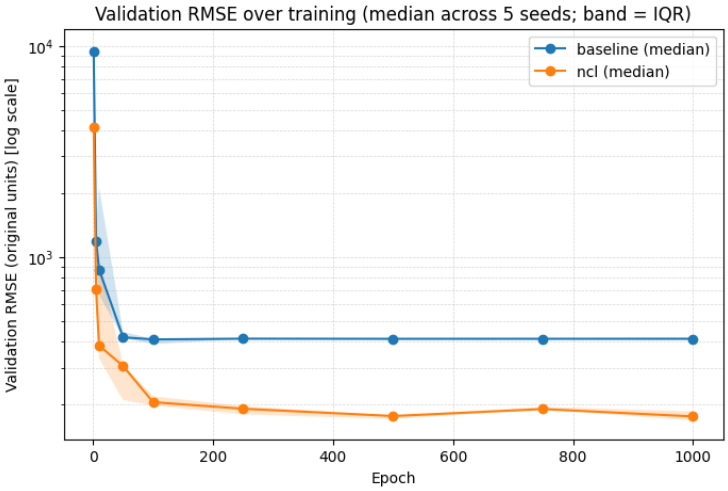

Abbildung 4.6: Logarithmisch skalierte Median-Validierungs-RMSE von Basis- und NCL-Modell über Epochen und Läufe

VALIDIERUNGS-RMSE UND -KOHÄRENZ Das folgende Streudiagramm fasst die Ergebnisse der Kohärenz, der Validierungs-RMSE und der Robustheit zusammen. Deutlich wird die Verbesserung aller angegebenen Metriken sowie die reduzierte Wertstreuung über Seeds.

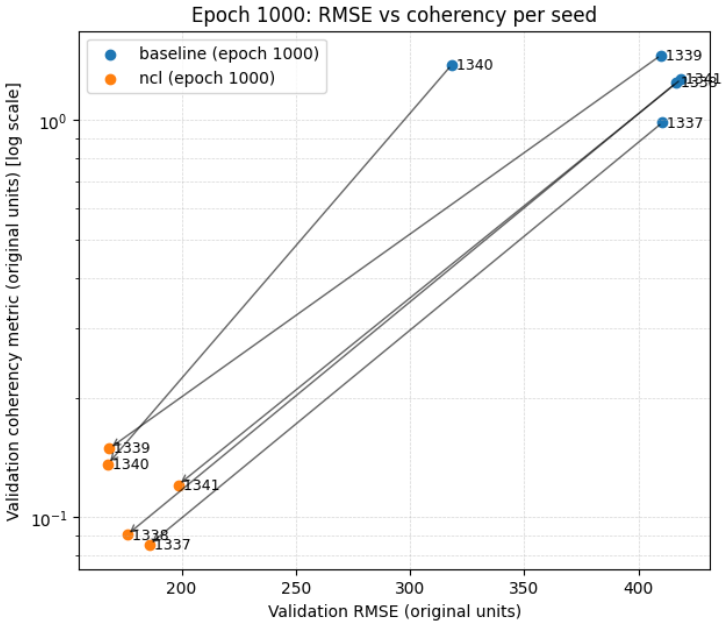

Abbildung 4.7: Logarithmische Validierungs-Kohärenz gegen Validierungs-RMSE in Originalwerten

4.1 DISKUSSION

Die Diskussion ordnet die Ergebnisse intern (Basismodell im Vergleich zu NCL) und extern im Vergleich zu Hensgen ein. Anschließend werden Bezug zur Forschungsfrage, Limitationen und ein Ausblick formuliert.

4.1.1 *Zusammenfassung der Ergebnisse*

Über fünf Seeds zeigt NCL im Mittel Verbesserungen der Kohärenz, des RMSE und der Prognosegütevarianz auf Validierungs- und Testdaten gegenüber einem Basismodell mit reiner MSE-Verlustfunktion. Die Test-Kohärenz verbessert sich im Mittel um 92.4 % (Standardabweichung: 2.6 %), der Test-RMSE um 34.0 % (Standardabweichung: 20.2 %). Ähnliche Steigerungen zeigen sich bei der Validierung: Der Best-Val-RMSE verbessert sich um 52.6 % bei einer Standardabweichung von 15 % (vgl. Tabelle 4.1). Die Kohärenzverbesserung tritt früher im Training auf und bleibt über die Epochen stabil (vgl. Abbildung 4.3). Die Varianzreduktion beim Test-RMSE legt nahe, dass die Regularisierung auch die Ergebnisrobustheit erhöht.

4.1.2 *Interner Datenvergleich*

KOHÄRENZ-GENAUIGKEIT WECHSELWIRKUNG In den vorliegenden Ergebnissen zeigt sich ein positiver Einfluss von NCL. Ein einzelner Seed weist zwar einen schlechteren Test-RMSE unter NCL auf (Tabelle 4.2), allerdings verbessert sich die Kohärenz dafür.

STABILITÄT ÜBER SEEDS Das Basismodell zeigt eine hohe Streuung (z. B. zwei Seeds mit sehr hohen Fehlern), während NCL in einem engen Leistungsband bleibt (Abbildung 4.7). Dies spricht dafür, dass die Regularisierung den Optimierer in Regionen des Parameterraums führt, die zugleich kohärenter und stabiler generalisieren. Da NCL durch die eingebettete Strukturinformation einen induktive Verzerrung einführt, ist zu hinterfragen, ob dieser generalisierbar positive Auswirkungen auf die Prognosegüte hat. Für eine belastbare empirische Aussage ist $n = 5$ jedoch nicht ausreichend.

TRAININGSDYNAMIK UND ROLLE VON α Der frühe Einbruch der Kohärenzmetrik unter NCL ist erwartbar, da der Regularisierungsterm direkt kohärente Basisprognosen ausgeben soll. Der beobachtete α-Verlauf (vgl. A.2) zeigt, dass die Gewichtung nicht konstant bleibt, sondern phasenweise nachjustiert wird. Methodisch ist dies ein Vorteil (automatische Skalierung), kann jedoch auch eine Fehlerquelle sein. Ohne geeignete Vorverarbeitung/Skalierung kann α numerisch ungünstige Größenordnungen annehmen und damit die Balance der Terme destabilisieren (vgl.A.5). Daher wird für NCL mindestens eine Datennormalisierung vorausgesetzt.

4.1.3 *Externer Vergleich mit Hensgen*

Hensgen berichtet für den Tourism-Datensatz, dass NCL bei geeigneter Parametrisierung, sowohl die Genauigkeit als auch die Kohärenz verbessert. Insbesondere wird für eine Konfiguration mit $\alpha = 100$ und kleinem Inkrement $\alpha_i = 0.1$ eine Verbesserung der Test-RMSE gegenüber dem Basismodell beobachtet bei gleichzeitig verbesserter Kohärenz. [11]

Die vorliegenden Resultate replizieren die Richtung dieser Befunde, obwohl ein Feed-Forward-basiertes Grundmodell eingesetzt wird. Damit stützt das Experiment die zentrale Generalisierbarkeitsannahme. Solange eine lineare Ausgabeschicht vorhanden ist, kann NCL als Netzwerk-Regularisierung auch in anderen ANNs wirksam sein. Inhaltlich ist die Vergleichbarkeit der absoluten Zahlen jedoch eingeschränkt, weil unterschiedliche Modellarchitekturen, unterschiedliche Skalierungs-/Auswertungsentscheidungen sowie unterschiedliche Implementierungsdetails (z. B. Batch Normalization, Encoder/ Decoder-Varianten) die Metrikskalen beeinflussen können. Der relevante Punkt ist daher nicht die Übereinstimmung der absoluten Werte, sondern die konsistente Verbesserungstendenz (bessere Kohärenz und mindestens konkurrenzfähige Genauigkeit, nebenbei bessere Robustheit der Prognosegüte)

4.1.4 *Bezug auf die Ziele, Fragestellungen und Hypothesen*

Die Arbeit verfolgt zwei Ziele: die Übertragung von NCL auf ein Feed-Forward-basiertes Prognosemodell (N-BEATS) unter Nutzung des Tourismusdatensatzes [1, 2], um eine Modellagnostik von NCL zu stützen oder zu widerlegen, sowie die Analyse der NCL-Wirkung auf Kohärenz und Prognosegüte. Beide Ziele werden in der betrachteten Testumgebung erreicht. Die Experimente zeigen, dass NCL auch in einem rein feed-forward organisierten Modell wirksam ist und damit nicht an rekurrente Architekturen gebunden zu sein scheint. [11]

F1 (KOHÄRENZ) Die Forschungsfrage 1.0.4, ob NCL die hierarchische Kohärenz gegenüber einem Kontrollmodell ohne NCL verbessert, wird klar bejaht. Über alle Seeds sinkt die Test-Kohärenz von 1.747 ± 0.383 (Basismodell) auf 0.133 ± 0.034 (NCL), was einer relativen Verbesserung von ca. 92.4% entspricht. Auf Einzellaufebene verbessert sich die Kohärenz konsistent in allen 5/5 Fällen (mindestens 88.89%).

F2 (PROGNOSEGÜTE) Die Forschungsfrage 1.0.4 nach dem Einfluss von NCL auf die Prognosegüte (gemessen via Test-RMSE) wird ebenfalls bejaht: Im Mittel sinkt die Test-RMSE von 373.40 ± 114.14 auf 246.44 ± 6.29 (relative Verbesserung ca. 34.0%). Die Einzelläufe zeigen dabei 4/5 Verbesserungen. Seed 1340 ist ein negativer Ausreißer mit $\Delta\text{RMSE} = -7.44\%$. Auch das best-valRoot Mean Squared Error (RMSE) sinkt (von 325.54 ± 102.92 auf 154.45 ± 3.60), was auf eine günstigere Trainingsdynamik und eine frühere/-verlässlichere Erreichung der Generalisierung hindeutet.

ROBUSTHEIT Nebenbei wurde die Hypothese, dass NCL robustere Prognosen erzeugt, in dieser Testumgebung durch die reduzierte Streuung über Seeds gestützt: Die Standardabweichung der Test-RMSE fällt von 114.14 (Basismodell) auf 6.29 (NCL), d. h. die Ergebnisse sind weniger sensitiv gegenüber der zufälligen Initialisierung.

F3 (VERGLEICH ZU HENSGEN) Im externen Vergleich 1.0.4 zeigt sich ein konsistentes Bild: Für den Tourism-Datensatz berichtet Hensgen eine relative Verbesserung von $\Delta RMSE = 35.47\%$ und Δ Kohärenz$= 86.44\%$ gegenüber dem Kontrollmodell (vgl. [11], 2.2).

Die vorliegenden N-BEATS Ergebnisse liegen hinsichtlich Richtung und Größenordnung der Effekte in einem vergleichbaren Bereich zu Hengsens Resultaten - insbesondere zeigen sich Kohärenzgewinne bei gleichzeitig verbesserter Prognosegüte -, obwohl eine andere Architekturfamilie und eine größere Datensatzvariante verwendet wurden. Damit stützen die Resultate die zentrale Arbeitsthese, dass NCL als weiche Regularisierung über die letzte lineare Ausgabeschicht modellagnostisch wirksam sein kann, sofern die NCL-Voraussetzungen (letzte lineare Schicht, hierarchische Summenstruktur) erfüllt sind.

4.2 LIMITIERUNGEN

- **Ein-Schritt-Prognose**: Es wird ausschließlich $h = 1$ betrachtet. Für weitere Tests müssten mehr Ressourcen (stärkerer Prozessor und Grafikkarte) verwendet werden. Mehrschrittige Prognosen (direkt oder rekursiv) bleiben offen.
- **Begrenzte Hyperparameter-Suche**: Zur Vergleichbarkeit mit Hensgens Protokoll wird kein umfangreiches Tuning durchgeführt. Stabilität wird über feste Seeds und feste Hyperparameter erreicht. Dies dient der Reproduzierbarkeit, Transparenz und vergleichbar zu Hensgen. Jedoch ist zu kritisieren, ob eine direkte Übertragung der Hyperparametern von einem Long-Short-Term-Memory Modell, ähnliche Wirkungen auf ein N-BEATS Modell besitzen würde.
- **Unvollständige Spezifikation**: Einige Implementierungsdetails aus Hensgens Versuchsaufbau können nicht vollständig rekonstruiert werden. Der verwendete Code ist nicht veröffentlicht worden. Daher wurden auf Basis gängiger Praxis, ähnlich zu [28, Tab.1] synthetisiert und implementiert. Die Annahme ist, dass Hensgen auch nach einer gängigen Praxis seinen Code Implementiert hat. Die Vergleichbarkeit leidet unter diesen Annahmen entwickelte Implementierung.
- **Rechenressourcen**: Lange Trainingsläufe (1000 Epochen) sind rechenintensiv. Erweiterungen (z. B. größere Modelle, Multi-Step, zusätzliche Basismodelle, mehr α und α_i Werte, vergleiche mit anderen Kohärenzabgleichsmethoden) werden auf zukünftige Forschung verschoben. Des weiteren wurde keine Laufzeitgegenüberstellung erstellt.
- **α-Rate:** Hensgen evaluiert mehrere α-Werte und Inkrementstrategien α_i. Aus Umfangs-gründen wird in dieser Arbeit nur die für den Australian-Tourism-Datensatz in Hensgens Experimenten beste α-Konfiguration übernommen und getestet. Hensgen berichtet hierfür die beste Leistung bei $\alpha = 100$ mit Inkrementen von 0.1 (vgl.[11, S.25]).
- **Seed-Anzahl:** Fünf Seeds erlauben eine Tendenzaussage, aber nur eingeschränkt belastbare Inferenzstatistik.
- **Verwendete Metriken**: Aus Umfangsgründen können keine weiteren Metriken getestet werden. Obwohl diese nach 2.4.2 ersetzt werden können für andere Fehlermaße. Explizit wird dazu auf die Diskussion zur Verwendbarkeit von MSE, RMSE und von anderen geeigneten Fehlermaßen für hierarchische Zeitreihen von Stephan Kolassa referenziert, (vgl. [22]).
- Es wird ein **N-BEATS-Derivat** benutzt. Dieses unterscheidet sich zu der Originalimplementierung zu [27] (Blockstruktur mit expliziter Backcast/ Forecast-Zerlegung). Damit wird primär getestet, ob NCL unabhängig vom Grundmodell funktioniert, nicht ob die N-BEATS Leistung ausgeschöpft werden kann.

5 FAZIT

Diese Arbeit überträgt die NCL auf ein rein feed-forward-basiertes Prognosemodell in Form eines N-BEATS-Derivats unter Nutzung des Australian-Tourism-Datensatzes nach Hensgen und untersucht die Wirkung auf Kohärenz und Prognosegüte. In der betrachteten Testumgebung ist NCL wirksam. Dies stützt die Annahme, dass NCL nicht an rekurrente Architekturen gebunden ist, sofern eine letzte lineare Ausgabeschicht und eine additive Summenstruktur vorliegen.

Die Forschungsfrage zur Kohärenz wird bejaht. Über alle Seeds sinkt die Test-Kohärenz von $Kohrenz_{\mathrm{B}} = 1.747 \pm 0.383$ im Basismodell auf $Kohrenz_{\mathrm{NCL}} = 0.133 \pm 0.034$ im NCL-Modell. Dies entspricht einer relativen Verbesserung von 92.4%. Auf Einzellaufebene verbessert sich die Kohärenz in 5/5 Fällen, die minimale Verbesserung beträgt 88.89%.

Die Forschungsfrage zur Prognosegüte wird ebenfalls bejaht. Im Mittel sinkt die Test-RMSE von $RMSE_{\mathrm{B}} = 373.40 \pm 114.14$ auf $RMSE_{\mathrm{NCL}} = 246.44 \pm 6.29$, entsprechend einer relativen Verbesserung von 34.0%. Die Einzelläufe zeigen Verbesserungen in 4/5 Fällen, Seed 1340 ist ein negativer Ausreißer mit $\Delta RMSE = -7.44\%$. Zusätzlich sinkt die Best-Val-RMSE von 325.54 ± 102.92 auf 154.45 ± 3.60.

Die reduzierte Streuung über Seeds stützt die Hypothese einer erhöhten Robustheit. Die Standardabweichung der Test-RMSE fällt von 114.14 im Basismodell auf 6.29 im NCL-Modell.

Im externen Vergleich ergibt sich ein konsistentes Bild. Hensgen berichtet für den Tourism-Datensatz eine relative RMSE-Verbesserung von 35.47% sowie eine Kohärenzverbesserung von 86.44% gegenüber dem Basismodell. Die hier erzielten Ergebnisse liegen hinsichtlich Richtung und Größenordnung in einem vergleichbaren Bereich. Insgesamt stützen die Resultate die Arbeitsthese, dass NCL als Soft-Regularisierung über die letzte lineare Ausgabeschicht modellagnostisch wirksam sein kann, sofern die strukturellen Annahmen erfüllt sind.

5.0.1 *Limitierungen*

Die experimentelle Untersuchung ist durch mehrere Designentscheidungen begrenzt. Es wird ausschließlich eine One-step-ahead-Punktprognose ($h = 1$) betrachtet; Aussagen zu mehrschrittigen Settings bleiben offen. Zur Vergleichbarkeit mit dem Referenzprotokoll erfolgt keine breite Hyperparametersuche, sodass unklar bleibt, ob übernommene Einstellungen (insbesondere aus dem LSTM-Kontext) für das verwendete N-BEATS-Setting gleichermaßen optimal sind. Zudem sind einzelne Details der Referenz nicht vollständig rekonstruierbar, da keine vollständige Implementierung öffentlich

verfügbar ist; die Umsetzung basiert daher auf gängiger Praxis und einer bestmöglichen Annäherung, was die Strenge direkter Vergleiche einschränkt. Rechenressourcen begrenzen Umfang und Tiefe der Experimente: Trainings über 1000 Epochen sind kostenintensiv, weshalb keine größeren Modelle, wie zusätzliche Basismodelle, umfangreichen $(\alpha) - (\alpha_i)$-Varianten, Vergleiche mit harten Abgleichmethoden oder eine systematische Laufzeitanalyse durchgeführt werden. Entsprechend wird nur eine (α)-Konfiguration aus der Referenz getestet ($(\alpha = 100), (\alpha_i)$-Inkremente 0.1), ohne Sensitivitätsanalyse. Fünf Seeds erlauben eine robuste Tendenzaussage, liefern aber nur eingeschränkt belastbare Inferenz über die Varianz stochastischer Trainingsläufe. Auch die Auswertung bleibt aus Umfangsgründen auf MSE, RMSE und die definierte Kohärenzmetrik beschränkt; skalenfreie oder wettbewerbsorientierte Fehlermaße werden nicht betrachtet. Schließlich basiert die Umsetzung auf einem N-BEATS-Derivat, das von der Originalarchitektur abweicht, sodass primär die Modellunabhängigkeit von NCL und nicht die maximal mögliche Leistung einer originalgetreuen N-BEATS-Implementierung untersucht wird.

6 WEITERE FORSCHUNG

Aus den genannten Limitierungen ergeben sich Ansatzpunkte für zukünftige Arbeiten. Dazu zählen Mehrschrittprognosen, breiteres Hyperparameter-Tuning, zusätzliche Architekturen,

Vergleiche mit Projektions- und MinT-Verfahren, Laufzeitanalysen sowie robustere statistische Absicherungen über mehr Seeds und ergänzende Metriken. Möglich ist auch die α-Raten-Optimierung dynamischer zu gestalten.

WEITERE ANSÄTZE Ein Ansatz könnte ein *lateraler Abgleich* über mehrere Modelle hinweg sein. Während klassische Ansätze den Abgleich typischerweise innerhalb eines einzelnen Prognosemodells formulieren, erscheint es naheliegend, Kohärenzbedingungen als allgemeines Regularisierungskonzept zu interpretieren, das unterschiedliche Modellklassen gemeinsam adressieren kann. Dies eröffnet die Frage, ob sich kohärenzbasierte Regularisierung als modellübergreifendes Prinzip etablieren lässt - etwa indem verschiedene Prognosemodelle parallel trainiert und ihre Ausgaben in einem gemeinsamen kohärenten Raum gekoppelt werden. Methodisch würde dies die Brücke schlagen zwischen Ensemble-Ansätzen, Multi-Task-Learning und hierarchischer Restriktionsmodellierung.

Ein zweiter Block betrifft *temporale Kohärenzbildung*. Die vorliegende Arbeit fokussiert ausschließlich auf hierarchische Aggregationsbeziehungen zu einem Zeitpunkt; in der Literatur findet sich jedoch auch die temporale Kohärenzbildung (vgl. [3]). In der Praxis können zusätzlich zeitliche Konsistenzbedingungen von Interesse sein, etwa Periodensummen, Inventarflussgleichungen oder Bilanzidentitäten zwischen Zeitpunkten. Es wäre daher naheliegend, Hensgens Grundidee von NCL auf temporale Restriktionen zu erweitern, um Kohärenz nicht nur über Hierarchieebenen, sondern auch über den Zeitverlauf weich zu regulieren. Daraus ergäben sich neue Fragestellungen zur Stabilität mehrschrittiger Prognosen, zur Fehlerakkumulation und zur Gewichtung konkurrierender Restriktionen.

Darauf aufbauend erscheint ein Einsatz kohärenzbasierter Maße als *systemisches Beobachtungsinstrument* möglich. Kohärenzverletzungen lassen sich nicht nur als Modellfehler interpretieren, sondern auch als Indikator für Datenprobleme, Prozessbrüche oder nicht modellierte Effekte wie Meldeartefakte und Strukturänderungen. In dieser Perspektive wäre Kohärenz nicht ausschließlich ein Optimierungsziel, sondern auch ein diagnostisches Signal zur Überwachung von Datenpipelines und operativen Systemen - insbesondere in Anwendungen mit Buchhaltungsbeziehungen oder physikalischen Flussgleichungen.

Ein weiterer Ansatz betrifft die *Datenstrukturanalyse hierarchischer Datensätze*, die einen erheblichen Anteil am Gesamtaufwand einnehmen kann.

Daraus ergibt sich als pragmatische Entwicklungsrichtung der Aufbau von Werkzeugen zur schnelleren Analyse und Validierung von Summenstrukturen, Aggregationspfaden, Ebenendefinitionen und Konsistenzchecks. Solche Werkzeuge könnten Reproduzierbarkeit und Vergleichbarkeit verbessern, indem sie Hierarchien automatisiert rekonstruieren, Summenmatrizen erzeugen, Kohärenztests standardisieren und typische Fehlerquellen in Datenaufbereitung und Evaluation frühzeitig sichtbar machen.

Schließlich betrifft ein letzter Ansatz die *Kombination von NCL mit alternativen Verlustfunktionen*. In dieser Arbeit wurde die Prognosegüte ausschließlich über quadratische Fehlermaße bewertet. Aus theoretischer Sicht ist jedoch zu erwarten, dass unterschiedliche Domänen und Fehlerstrukturen unterschiedliche Verlustfunktionen bevorzugen – etwa robustere Maße bei Ausreißern oder asymmetrische Kostenfunktionen bei domänenspezifischen Zielsetzungen. Daraus folgt die Frage, wie sich NCL mit anderen Verlustfamilien kombinieren lässt und ob sich ein systematisches Vorgehen ableiten lässt, das Basisverlustfunktion und Kohärenzregularisierung gemeinsam domänenspezifisch begründet.

Teil I

APPENDIX

A

ANHANG

A.1 GITREPOSITORY

Link zum Git-repository mit Implementierungscode, Quelldaten, Datenanalyse und Erdgebissdaten aller Runs Fehlversuche: https://github.com/Gibbyace/Network-Coherency-Loss-on-Australian-Tourism-Data-via-N-BEATS-Thesis

A.2 ROHDATEN UND WEITERE PLOTS

	mode	test_mse	test_mse	test_rmse	test_rmse	test_coh	test_coh	best_val_rmse
1		mean	std	mean	std	mean	std	mean
2	baseline	149847.96972972975	80791.11944862698	373.39826295209343	114.13646959938481	1.747231924533844	0.38297387699716023	325.5423321888054
3	ncl	60763.73285285286	3136.3372687146384	246.43885985827973	6.2870108760061125	0.13337353020906448	0.03371235571514728	154.4499706116873

Abbildung A.1: Zusammenfassung der Ergebnisse als Mittelwert und Standardabweichung

	mode	seed	best_val_rmse	test_mse	test_rmse	test_coh	test_mse_scaled	test_rmse_scaled
1	baseline	1337	222.09654065151346	67342.25585585585	259.50386481872647	1.9684175252914429	0.045404973760381474	0.21308442871402283
2	baseline	1338	409.3933262372111	218335.75015015015	467.2641117720792	1.3895544409751892	0.05229546143127991 6	0.22868200941761885
3	baseline	1339	409.364819870672	216271.0774774775	465.0495430354464	2.1018943786621094	0.05229984317813908	0.2286915896532688
4	baseline	1340	205.26256321077125	57215.454654654655	239.19752225860253	1.2774285674095154	0.044836523153402426	0.211746365148029
5	baseline	1341	381.5944109738591	190075.31051051052	435.9762728756125	1.9988647103309631	0.05082681787622584	0.22544803808466785
6	ncl	1337	160.70374959886635	57998.80840840841	240.82941765575154	0.17739465832710266	0.04436600143845017	0.21063238459090322
7	ncl	1338	153.0357654717142	58861.883483483485	242.61468109634976	0.154337577521801	0.04467365390903598 6	0.2113614295680174
8	ncl	1339	151.80942818707436	60455.14474474474	245.87627934541538	0.0904797725379467	0.04461865811734586	0.21123129057349874
9	ncl	1340	152.49083008909506	66045.35975975975	256.99291772295936	0.11581433936953545	0.04441610112920537 5	0.210751277882734
10	ncl	1341	154.21007971168643	60457.46786786787	245.88100347092262	0.12884130328893661	0.0446834552753437	0.21138461456630117

Abbildung A.2: Ergebnisse des Basismodells und des NCL-Modells pro Lauf

α DYNAMIK Auffällig ist die ca. logarithmische Bewegung zwischen Epoche 0 und 100. Wobei diese Sprunghaft auf einen Wert über 120000 aufsteigt. Ab Epoche 100 senkt sich diese bis Epoche 1000 mit zwischenzeitlichen Aufwärtswellen. Möglicherweise liegt dies wegen dem Logarithmischen Anteil der Lernraten-Skalierung in 3.1 und durch die α_i Skalierung, dass diese erst ab 100 Epochen stark nachjustiert. Dies könnte darauf hindeuten,

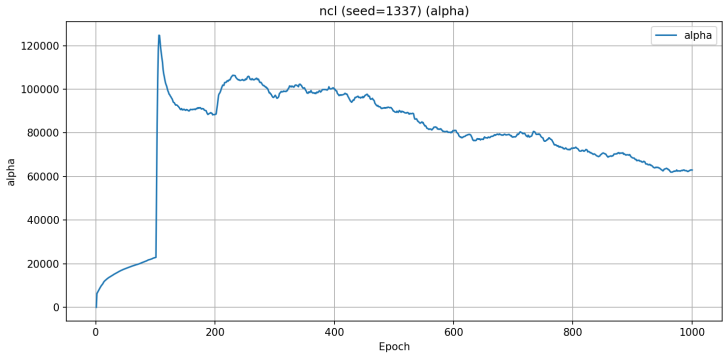

Abbildung A.3: α pro Epoche

A.3 BEISPIELRECHNUNG DER AUSWIRKUNG VON NORMALISIERUNG

Betrachtet wird ein Modell mit einem Trainingspunkt:

$$\hat{y} = wx, \qquad \mathcal{L} = (\hat{y} - y)^2.$$

Der Gradient nach w ist

$$\frac{\partial \mathcal{L}}{\partial w} = 2(\hat{y} - y)\, x.$$

Wähle $x = 50$, $y = 1000$, Start $w_0 = 0$ und Lernrate $\eta = 0.01$.

(A) Ohne Normalisierung

$$\begin{aligned}
\hat{y}_0 &= w_0 x = 0 \cdot 50 = 0, \\
e_0 &= \hat{y}_0 - y = 0 - 1000 = -1000, \\
\nabla_0 &= 2e_0 x = 2 \cdot (-1000) \cdot 50 = -100000, \\
w_1 &= w_0 - \eta \nabla_0 = 0 - 0.01 \cdot (-100000) = 1000.
\end{aligned}$$

Nächster Schritt (zeigt Instabilität):

$$\begin{aligned}
\hat{y}_1 &= w_1 x = 1000 \cdot 50 = 50000, \\
e_1 &= \hat{y}_1 - y = 50000 - 1000 = 49000, \\
\nabla_1 &= 2e_1 x = 2 \cdot 49000 \cdot 50 = 4900000, \\
w_2 &= w_1 - \eta \nabla_1 = 1000 - 0.01 \cdot 4900000 = -48000
\end{aligned}$$

Damit wäre bereits

$$\hat{y}_2 = w_2 x = -48000 \cdot 50 = -2400000,$$

also eine starke Divergenz.

(B) Mit Min–Max-Normalisierung (in $[0, 1]$)

. Exemplarische Skalierung:

$$x^{(s)} = \frac{x}{100} = 0.5, \qquad y^{(s)} = \frac{y}{2000} = 0.5$$

Trainiert wird im Normalisierung Raum:

$$\hat{y}^{(s)} = wx^{(s)}, \qquad \mathcal{L}^{(s)} = (\hat{y}^{(s)} - y^{(s)})^2,$$

mit Gradient

$$\frac{\partial \mathcal{L}^{(s)}}{\partial w} = 2(\hat{y}^{(s)} - y^{(s)})\, x^{(s)}$$

Für $w_0 = 0$ und $\eta = 0.01$:

$$\begin{aligned}
\hat{y}_0^{(s)} &= w_0 x^{(s)} = 0 \cdot 0.5 = 0, \\
e_0^{(s)} &= \hat{y}_0^{(s)} - y^{(s)} = 0 - 0.5 = -0.5, \\
\nabla_0^{(s)} &= 2e_0^{(s)} x^{(s)} = 2 \cdot (-0.5) \cdot 0.5 = -0.5, \\
w_1 &= w_0 - \eta \nabla_0^{(s)} = 0 - 0.01 \cdot (-0.5) = 0.005
\end{aligned}$$

Der nächste Schritt bleibt ebenfalls klein und stabil:

$$\begin{aligned}
\hat{y}_1^{(s)} &= 0.005 \cdot 0.5 = 0.0025, \\
e_1^{(s)} &= 0.0025 - 0.5 = -0.4975, \\
\nabla_1^{(s)} &= 2 \cdot (-0.4975) \cdot 0.5 = -0.4975, \\
w_2 &= 0.005 - 0.01 \cdot (-0.4975) = 0.009975
\end{aligned}$$

Kernaussage

Bei gleicher Lernrate ist die Schrittweite im Update proportional zur Gradientengröße. Ohne Normalisierung ist $|\nabla_0| = 100000$ (Update $\Delta w = 1000$), mit Normalisierung $|\nabla_0^{(s)}| = 0.5$ (Update $\Delta w = 0.005$). Normalisierung hält Gradienten und Updates in einem numerisch stabilen Bereich.

A.4 BEISPIELRECHNUNG VON NCL

Beispielrechnung für NCL im Gradientenverfahren Hierarchie: 2 disaggregierte Knoten (m=2) und 1 Aggregat (n=3): y = (Total, d1, d2)$^T mit Total = d1 + d2$.

$$S = \begin{bmatrix} 1 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \in \mathbb{R}^{3\times 2} \qquad (n = 3,\ m = 2).$$

$$S^+ = (S^\top S)^{-1} S^\top = \begin{bmatrix} \frac{1}{3} & \frac{2}{3} & -\frac{1}{3} \\ \frac{1}{3} & -\frac{1}{3} & \frac{2}{3} \end{bmatrix} \in \mathbb{R}^{2\times 3}.$$

$$A = SS^+ = \begin{bmatrix} \frac{2}{3} & \frac{1}{3} & \frac{1}{3} \\ \frac{1}{3} & \frac{2}{3} & -\frac{1}{3} \\ \frac{1}{3} & -\frac{1}{3} & \frac{2}{3} \end{bmatrix} \in \mathbb{R}^{3\times 3}.$$

$$W_L = \begin{bmatrix} 2 & 0 \\ 1 & 1 \\ 0 & 2 \end{bmatrix} \in \mathbb{R}^{3\times 2}, \qquad \mathbf{b}_L = \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix} \in \mathbb{R}^3.$$

$$AW_L = \begin{bmatrix} \frac{2}{3} & \frac{1}{3} & \frac{1}{3} \\ \frac{1}{3} & \frac{2}{3} & -\frac{1}{3} \\ \frac{1}{3} & -\frac{1}{3} & \frac{2}{3} \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 1 & 1 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} \frac{5}{3} & 1 \\ \frac{4}{3} & 0 \\ \frac{1}{3} & 1 \end{bmatrix}.$$

$$A\mathbf{b}_L = \begin{bmatrix} \frac{2}{3} & \frac{1}{3} & \frac{1}{3} \\ \frac{1}{3} & \frac{2}{3} & -\frac{1}{3} \\ \frac{1}{3} & -\frac{1}{3} & \frac{2}{3} \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix} = \begin{bmatrix} \frac{4}{3} \\ \frac{5}{3} \\ -\frac{1}{3} \end{bmatrix}.$$

$$W_L - AW_L = \begin{bmatrix} 2 & 0 \\ 1 & 1 \\ 0 & 2 \end{bmatrix} - \begin{bmatrix} \frac{5}{3} & 1 \\ \frac{4}{3} & 0 \\ \frac{1}{3} & 1 \end{bmatrix} = \begin{bmatrix} \frac{1}{3} & -1 \\ -\frac{1}{3} & 1 \\ -\frac{1}{3} & 1 \end{bmatrix}.$$

$$\mathbf{b}_L - A\mathbf{b}_L = \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix} - \begin{bmatrix} \frac{4}{3} \\ \frac{5}{3} \\ -\frac{1}{3} \end{bmatrix} = \begin{bmatrix} -\frac{1}{3} \\ \frac{1}{3} \\ \frac{1}{3} \end{bmatrix}.$$

$$\|W_L - AW_L\|_F = \sqrt{\left(\frac{1}{3}\right)^2 + (-1)^2 + \left(-\frac{1}{3}\right)^2 + (1)^2 + \left(-\frac{1}{3}\right)^2 + (1)^2} = \sqrt{\frac{10}{3}} = \frac{\sqrt{30}}{3}.$$

$$\|\mathbf{b}_L - A\mathbf{b}_L\|_2 = \sqrt{\left(-\frac{1}{3}\right)^2 + \left(\frac{1}{3}\right)^2 + \left(\frac{1}{3}\right)^2} = \sqrt{\frac{1}{3}} = \frac{\sqrt{3}}{3}.$$

$$\mathcal{L}_{\mathrm{NCL}} = \|W_L - AW_L\|_F + \|\mathbf{b}_L - A\mathbf{b}_L\|_2 = \frac{\sqrt{30}}{3} + \frac{\sqrt{3}}{3} = \frac{\sqrt{30}+\sqrt{3}}{3} \approx 2.403.$$

A.5 NCL OHNE NORMALISIERUNG

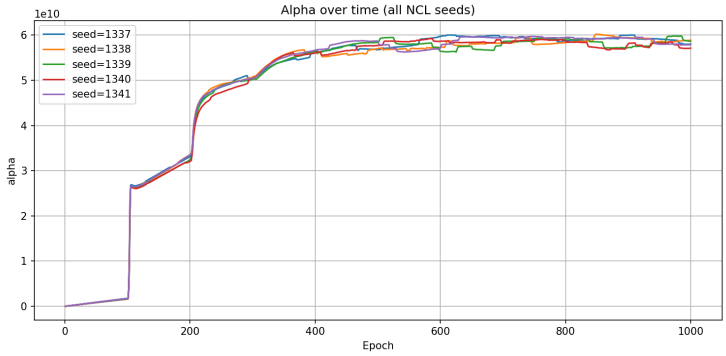

Abbildung A.4: Alphaskalierung über alle Seeds

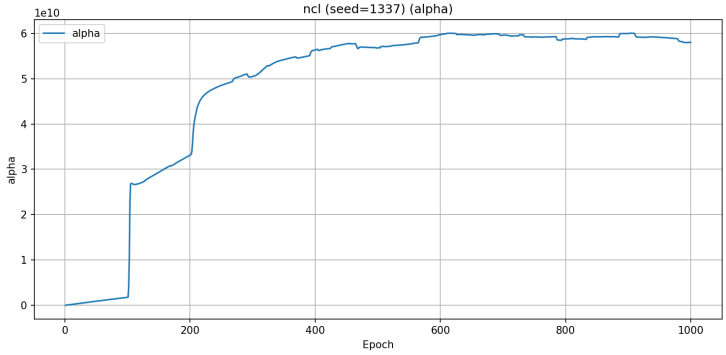

Abbildung A.5: Alphaskalierung über Seed 1337

Auffällig die hohe Alpha-rate von bis zu 6×10^{10}

LITERATUR

[1] Mahsa Ashouri, Rob J Hyndman und Galit Shmueli. "Fast Forecast Reconciliation Using Linear Models". In: *Journal of Computational and Graphical Statistics* 31.1 (2022), S. 263–282. DOI: 10.1080/10618600.2021.1939038. URL: https://doi.org/10.1080/10618600.2021.1939038.

[2] George Athanasopoulos, Roman A. Ahmed und Rob J. Hyndman. "Hierarchical forecasts for Australian domestic tourism". In: *International Journal of Forecasting* 25.1 (2009), S. 146–166. ISSN: 0169-2070. DOI: https://doi.org/10.1016/j.ijforecast.2008.07.004. URL: https://www.sciencedirect.com/science/article/pii/S0169207008000691.

[3] George Athanasopoulos, Rob J. Hyndman, Nikolaos Kourentzes und Anastasios Panagiotelis. "Forecast reconciliation: A review". In: *International Journal of Forecasting* 40.2 (2024), S. 430–456. ISSN: 0169-2070. DOI: https://doi.org/10.1016/j.ijforecast.2023.10.010. URL: https://www.sciencedirect.com/science/article/pii/S0169207023001097.

[4] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Information Science and Statistics. New York, NY: Springer, 2006. ISBN: 978-0-387-31073-2.

[5] Sven F. Crone. *Neuronale Netze zur Prognose und Disposition im Handel*. 1. Aufl. Bd. 60. Betriebswirtschaftliche Forschung zur Unternehmensführung. Wiesbaden: Gabler Verlag, 2010, S. XXXVI, 488. ISBN: 978-3-8349-8631-3. DOI: 10.1007/978-3-8349-8631-3.

[6] Dudenredaktion. *Prognose*. Duden online (Online-Wörterbuch). URL: https://www.duden.de/rechtschreibung/Prognose (besucht am 01. 03. 2026).

[7] Ian Goodfellow, Yoshua Bengio und Aaron Courville. *Deep Learning*. MIT Press, 2016. URL: http://www.deeplearningbook.org.

[8] Stephen Haben, Marcus Voss und William Holderbaum. "Time Series Forecasting: Core Concepts and Definitions". In: *Core Concepts and Methods in Load Forecasting: With Applications in Distribution Networks*. Cham: Springer International Publishing, 2023, S. 55–66. ISBN: 978-3-031-27852-5. DOI: 10.1007/978-3-031-27852-5_5. URL: https://doi.org/10.1007/978-3-031-27852-5_5.

[9] Mohsen Hamoudia, Spyros Makridakis und Evangelos Spiliotis. *Forecasting with Artificial Intelligence. Theory and Applications*. 1. Aufl. Palgrave Advances in the Economics of Innovation and Technology. Palgrave Macmillan Cham, 2023, S. XLIV, 412. ISBN: 978-3-031-35879-1. DOI: 10.1007/978-3-031-35879-1.

[10] Xing Han, Sambarta Dasgupta und Joydeep Ghosh. "Simultaneously Reconciled Quantile Forecasting of Hierarchically Related Time Series". In: *Proceedings of The 24th International Conference on Artificial Intelligence and Statistics*. Hrsg. von Arindam Banerjee und Kenji Fukumizu. Bd. 130. Proceedings of Machine Learning Research. PMLR, 2021, S. 190–198. URL: https://proceedings.mlr.press/v130/han21a.html.

[11] Michael Lowell Hensgen. "Coherency Loss for Hierarchical Time Series Forecasting". Magisterarb. Cambridge, MA, USA: Massachusetts Institute of Technology, Mai 2024. URL: https://hdl.handle.net/1721.1/156799.

[12] Hansika Hewamalage, Christoph Bergmeir und Kasun Bandara. "Recurrent Neural Networks for Time Series Forecasting: Current status and future directions". In: *International Journal of Forecasting* 37.1 (2021), S. 388–427. ISSN: 0169-2070. DOI: https://doi.org/10.1016/j.ijforecast.2020.06.008. URL: https://www.sciencedirect.com/science/article/pii/S0169207020300996.

[13] Sepp Hochreiter und Jürgen Schmidhuber. "Long Short-Term Memory". In: *Neural Computation* 9.8 (1997), S. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.

[14] Kurt Hornik, Maxwell Stinchcombe und Halbert White. "Multilayer feedforward networks are universal approximators". In: *Neural Networks* 2.5 (1989), S. 359–366. ISSN: 0893-6080. DOI: https://doi.org/10.1016/0893-6080(89)90020-8. URL: https://www.sciencedirect.com/science/article/pii/0893608089900208.

[15] Rob J. Hyndman, Roman A. Ahmed, George Athanasopoulos und Han Lin Shang. "Optimal combination forecasts for hierarchical time series". In: *Computational Statistics Data Analysis* 55.9 (2011), S. 2579–2589. ISSN: 0167-9473. DOI: https://doi.org/10.1016/j.csda.2011.03.006. URL: https://www.sciencedirect.com/science/article/pii/S0167947311000971.

[16] Rob J. Hyndman und George Athanasopoulos. *Forecasting: Principles and Practice*. 3rd. Accessed on 28 January 2026. Melbourne, Australia: OTexts, 2021. URL: https://OTexts.com/fpp3.

[17] Sergey Ioffe und Christian Szegedy. "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift". In: *Proceedings of the 32nd International Conference on Machine Learning*. Hrsg. von Francis Bach und David Blei. Bd. 37. Proceedings of Machine Learning Research. Lille, France: PMLR, 2015, S. 448–456. URL: https://proceedings.mlr.press/v37/ioffe15.html.

[18] Harshavardhan Kamarthi, Lingkai Kong, Alexander Rodríguez, Chao Zhang und B Aditya Prakash. "PROFHIT: Probabilistic Robust Forecasting for Hierarchical Time-series". In: *arXiv preprint arXiv:2206.07940* (2022). URL: https://github.com/AdityaLab/PROFHiT.

[19] Mateusz Kasprzyk, Paweł Pełka, Boris N. Oreshkin und Grzegorz Dudek. "Enhanced N-BEATS for mid-term electricity demand forecasting". In: *Applied Soft Computing* 182 (2025), S. 113575. ISSN: 1568-4946. DOI: https://doi.org/10.1016/j.asoc.2025.113575. URL: https://www.sciencedirect.com/science/article/pii/S1568494625008865.

[20] Yang-Seon Kim, Moon Keun Kim, Nuodi Fu, Jiying Liu, Junqi Wang und Jelena Srebric. "Investigating the impact of data normalization methods on predicting electricity consumption in a building using different artificial neural network models". In: *Sustainable Cities and Society* 118 (2025), S. 105570. ISSN: 2210-6707. DOI: https://doi.org/10.1016/j.scs.2024.105570. URL: https://www.sciencedirect.com/science/article/pii/S2210670724003962.

[21] Diederik P. Kingma und Jimmy Ba. "Adam: A Method for Stochastic Optimization". In: *Proceedings of the International Conference on Learning Representations (ICLR)*. Accessed: 2026-02-10. 2015. URL: https://arxiv.org/abs/1412.6980.

[22] Stephan Kolassa. "Do we want coherent hierarchical forecasts, or minimal MAPEs or MAEs? (We won't get both!)" In: *International Journal of Forecasting* 39.4 (2023), S. 1512–1517. ISSN: 0169-2070. DOI: https://doi.org/10.1016/j.ijforecast.2022.11.006. URL: https://www.sciencedirect.com/science/article/pii/S0169207022001492.

[23] Jens-Peter Kreiß und Georg Neuhaus. *Einführung in die Zeitreihenanalyse*. 1. Aufl. Statistik und ihre Anwendungen. Heidelberg: Springer Berlin, 2006, S. XIV, 388. ISBN: 978-3-540-33571-9. DOI: 10.1007/3-540-33571-4.

[24] Robert Labus. *Pseudoinverse Matrizen: Moore–Penrose-Inverse*. Universität Kassel, lecture notes. Version: 17 Mar 2019. Accessed: 2026-02-10. 2019. URL: https://www.mathematik.uni-kassel.de/~labus/Labus_Pseudoinverse2019.pdf.

[25] Víctor Manuel Landassuri-Moreno, Carmen L. Bustillo-Hernández, José Juan Carbajal-Hernández und Luis P. Sánchez Fernández. "Single-Step-Ahead and Multi-Step-Ahead Prediction with Evolutionary Artificial Neural Networks". In: *Progress in Pattern Recognition, Image Analysis, Computer Vision, and Applications*. Hrsg. von José Ruiz-Shulcloper und Gabriella Sanniti di Baja. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, S. 65–72. ISBN: 978-3-642-41822-8. DOI: https://doi.org/10.1007/978-3-642-41822-8_9.

[26] Spyros Makridakis, Evangelos Spiliotis und Vassilios Assimakopoulos. "The M4 Competition: 100,000 time series and 61 forecasting methods". In: *International Journal of Forecasting* 36.1 (2020). M4 Competition, S. 54–74. ISSN: 0169-2070. DOI: https://doi.org/10.1016/j.ijforecast.2019.04.014. URL: https://www.sciencedirect.com/science/article/pii/S0169207019301128.

[27] Boris N. Oreshkin, Dmitri Carpov, Nicolas Chapados und Yoshua Bengio. "N-BEATS: Neural basis expansion analysis for interpretable time series forecasting". In: *International Conference on Learning Representations*. 2020. URL: https://openreview.net/forum?id=r1ecqn4YwB.

[28] Gašper Petelin, Gjorgjina Cenikj und Tome Eftimov. "Towards understanding the importance of time-series features in automated algorithm performance prediction". In: *Expert Systems with Applications* 213 (2023), S. 119023. ISSN: 0957-4174. DOI: https://doi.org/10.1016/j.eswa.2022.119023. URL: https://www.sciencedirect.com/science/article/pii/S0957417422020413.

[29] PyTorch Contributors. *torch.optim.Adam*. PyTorch 2.4 Documentation. Accessed: 2026-02-10. 2023. URL: https://docs.pytorch.org/docs/2.4/generated/torch.optim.Adam.html.

[30] PyTorch Team. *PyTorch*. https://pytorch.org. URL: https://pytorch.org.

[31] Syama Sundar Rangapuram, Lucien D Werner, Konstantinos Benidis, Pedro Mercado, Jan Gasthaus und Tim Januschowski. "End-to-End Learning of Coherent Probabilistic Forecasts for Hierarchical Time Series". In: *Proceedings of the 38th International Conference on Machine Learning*. Hrsg. von Marina Meila und Tong Zhang. Bd. 139. Proceedings of Machine Learning Research. PMLR, 2021, S. 8832–8843. URL: https://proceedings.mlr.press/v139/rangapuram21a.html.

[32] Rob.J.Hyndman, Shanika L Wickramasuriya und George Athanasopoulos. *Australian Dataset Link*. URL: https://robjhyndman.com/publications/mint/.

[33] Jan Sellner. *Fundamentals of floating-point numbers and mixed precision training*. Blog. 3. Okt. 2024. URL: https://www.jansellner.net/blog/Fundamentals_of_floating-point_numbers_and_mixed_precision_training.

[34] Evangelos Spiliotis, Andreas Kouloumos, Vassilios Assimakopoulos und Spyros Makridakis. "Are forecasting competitions data representative of the reality?" In: *International Journal of Forecasting* 36.1 (2020). M4 Competition, S. 37–53. ISSN: 0169-2070. DOI: https://doi.org/10.1016/j.ijforecast.2018.12.007. URL: https://www.sciencedirect.com/science/article/pii/S0169207019300159.

[35] Juan Terven, Diana-Margarita Cordova-Esparza, Julio-Alejandro Romero-González, Alfonso Ramírez-Pedraza und E A Chávez-Urbiola. "A comprehensive survey of loss functions and metrics in deep learning". In: *Artificial Intelligence Review* 58.7 (2025), S. 95. DOI: https://doi.org/10.1007/s10462-025-11198-7. URL: https://link.springer.com/article/10.1007/s10462-025-11198-7.

[36] Jürgen Vogel. *Prognose von Zeitreihen. Eine Einführung für Wirtschaftswissenschaftler*. Wiesbaden: Springer Gabler, 2015. ISBN: 978-3-658-06837-0. DOI: 10.1007/978-3-658-06837-0.

[37] Xiaoqian Wang, Rob J. Hyndman und Shanika L. Wickramasuriya. "Optimal forecast reconciliation with time series selection". In: *European Journal of Operational Research* 323.2 (2025), S. 455–470. ISSN: 0377-2217. DOI: https://doi.org/10.1016/j.ejor.2024.12.004. URL: https://www.sciencedirect.com/science/article/pii/S0377221724009469.

[38] Shanika L. Wickramasuriya, George Athanasopoulos und Rob J. Hyndman. "Optimal Forecast Reconciliation for Hierarchical and Grouped Time Series Through Trace Minimization". In: *Journal of the American Statistical Association* 114.526 (2019), S. 804–819. DOI: 10.1080/01621459.2018.1448825. URL: https://doi.org/10.1080/01621459.2018.1448825.

[39] Klaus Wübbenhorst, Gustav A. Horn und Udo Kamps. *Prognose*. Gabler Wirtschaftslexikon. 2018. URL: https://wirtschaftslexikon.gabler.de/definition/prognose-43498/version-266827 (besucht am 24. 02. 2026).